

# Közelítő és szimbolikus számítások I.

Kidolgozott vizsgatételek

---

Átdolgozott, egységesített kiadás — az előadás fóliái alapján

29 kidolgozott tétel · Tartalomjegyzékkel

# Tartalomjegyzék

---

|                                                                                                    |    |
|----------------------------------------------------------------------------------------------------|----|
| 1. A számítógépes hiba különböző fajtái . . . . .                                                  | 4  |
| 2. Számítógépes számábrázolás és kerekítés . . . . .                                               | 6  |
| 3. Numerikus stabilitás, a hiba továbbterjedése, kondíciós szám . . . . .                          | 8  |
| 4. Intervallum aritmetika . . . . .                                                                | 10 |
| 5. Automatikus differenciálás . . . . .                                                            | 12 |
| 6. Lineáris egyenletrendszerek megoldása Gauss eliminációval . . . . .                             | 14 |
| 7. LU felbontás, eliminációs mátrixok, főelemkiválasztás . . . . .                                 | 17 |
| 8. Mátrixinvertálás, mátrixnormák, mátrixok kondíciós száma . . . . .                              | 20 |
| 9. Cholesky felbontás, QR ortogonális felbontás . . . . .                                          | 23 |
| 10. Mátrixok sajátértékei, sajátvektorai, a sajátértékek korlátai, a hatványmódszer . . . . .      | 26 |
| 11. A mátrixok típusai, hasonlósági transzformáció . . . . .                                       | 29 |
| 12. A sajátértékek és a sajátvektorok kondicionáltsága . . . . .                                   | 32 |
| 13. LR transzformáció . . . . .                                                                    | 34 |
| 14. Lineáris egyenletrendszerek iterációs módszerei, a Jacobi iteráció . . . . .                   | 36 |
| 15. Lineáris egyenletrendszerek iterációs módszerei konvergenciája . . . . .                       | 39 |
| 16. A Gauss–Seidel iteráció, mátrixok reguláris szétvágásai . . . . .                              | 41 |
| 17. Konjugált gradiens módszer . . . . .                                                           | 44 |
| 18. Lineáris egyenletek iterációs módszerei a MATLAB-ban . . . . .                                 | 47 |
| 19. Polinomok zérushelyei, Horner-elrendezés, Ruffini-sorozat, iterált Horner-elrendezés . . . . . | 50 |
| 20. Polinomok kezelése a MATLAB-ban . . . . .                                                      | 53 |
| 21. Függvényközelítések, Lagrange-interpoláció . . . . .                                           | 56 |
| 22. A Lagrange-interpoláció hibája, interpoláció a MATLAB-ban . . . . .                            | 58 |
| 23. Newton-módszer, szelőmódszer, húrmódszer . . . . .                                             | 60 |
| 24. Legkisebb négyzetek módszere . . . . .                                                         | 63 |
| 25. Spline-közelítés, megvalósítása a MATLAB-ban . . . . .                                         | 66 |
| 26. Numerikus integrálás, kvadrátúra-formulák . . . . .                                            | 69 |
| 27. Interpolációs kvadrátúra-formulák, ezek hibája . . . . .                                       | 72 |
| 28. Véges differenciák, Newton-Cotes formulák . . . . .                                            | 75 |

|                                                 |    |
|-------------------------------------------------|----|
| 29. Numerikus integrálás a MATLAB-ban . . . . . | 78 |
|-------------------------------------------------|----|

# 1. A számítógépes hiba különböző fajtái

A numerikus számítások egyik alapkérdése, hogy a számítógéppel előállított eredmény mennyire tér el a keresett pontos értéktől. A gyakorlatban szinte sosem a pontos értéket kapjuk meg, hanem annak egy **közelítését**, ezért a hiba nagyságának becslése a numerikus matematika központi feladata.

## A hiba fogalma és mérése

Legyen  $x$  a keresett, kiszámítandó pontos érték, és  $\tilde{x}$  ennek egy becslése (közelítése). A becslés jóságát többféle mérőszámmal jellemezzük:

- **Hiba (előjeles eltérés):** az  $x - \tilde{x}$  érték.
- **Abszolút hiba:** az  $|x - \tilde{x}|$  nemnegatív szám. Ennek egy felső korlátját **abszolút hibakorlátnak** nevezzük; a lehetséges korlátok közül a legkisebb a leginkább használható.
- **Relatív hiba:** az eltérést a pontos érték nagyságrendjéhez viszonyítja, felső korlátja a **relatív hibakorlát** (gyakran százalékban vagy ezrelékben adják meg).
- **A pontos (értékes) tizedesjegyek száma** a relatív hiba negatív tízesalapú logaritmus.

A négy legfontosabb képlet egységes jelöléssel:

$$x - \tilde{x} \quad |x - \tilde{x}| \quad \frac{|x - \tilde{x}|}{|x|} \quad -\log_{10} \frac{|x - \tilde{x}|}{|x|}$$

Azt mondjuk, hogy  $\tilde{x}$  az  $x$  egy század pontosságú becslése, ha 0,01 az abszolút hibakorlátja. Például a  $\pi = 3,14159265\dots$  becslése 3,141626 esetén az abszolút hiba  $\approx 0,000033$ , a relatív hiba  $\approx 0,000010$ , a pontos tizedesjegyek száma pedig  $\approx 4,98$  — összhangban azzal, hogy az első négy értékes jegy egyezik, az ötödik már nem.

## A hiba forrásai

A klasszikus hibaszámítás a számítási folyamatban három fő hibaforrást különböztet meg. (Ez az a felosztás, amelyet az előadás is használ — a köznapiban értelmében vett „hardveres” vagy „programozási hibák” nem ennek a numerikus osztályozásnak a részei.)

### 1. Öröklött (beviteli) hiba

Ha már a beolvasott, bemeneti adatok is pontatlanok, az ebből származó hibát **öröklött hibának** nevezzük. Az adatok jellemzően mérésből vagy külső forrásból származnak, és természetüknél fogva korlátozott pontosságúak; ez a pontatlanság az egész számítási folyamaton végigöröklődik.

*Példa:* ha a  $\pi$  értékét 3,14-nek vesszük a pontosabb 3,14159... helyett, akkor a további számítások már eleve terhelt adatból indulnak.

### 2. Képlethiba (módszer- vagy levágási hiba)

A számítási **módszer** elvi pontatlanságából származó eltérés. Tipikusan akkor lép fel, amikor egy végtelen folyamatot (sort, határértéket, folytonos modellt) véges sok lépéssel közelítünk, ezért szokás **levágási hibának** is nevezni.

*Példa:* egy Taylor-sor véges sok tagjával való közelítéskor a kihagyott tagok képlethibát okoznak; hasonlóan a deriváltat véges differenciával közelítő numerikus differenciálás is.

### 3. Kerekítési hiba

A kiszámított új számokat a gép csak véges sok lebegőpontos szám közül választva tudja ábrázolni; az ekkor elkövetett hiba a **kerekítési hiba**. Egyetlen kerekítés hibája kicsi, de sok művelet során a kerekítési hibák halmozódhatnak.

*Példa:* egyszeres pontosságú lebegőpontos szám csak mintegy 7–8 értékes tizedesjegyet tárol; ennél pontosabb eredményhez kettős pontosság szükséges.

## A hibaszámítás alapfeladata

A hibaszámítás alapfeladata: az öröklött hiba és a számítógépes eljárás ismeretében adjunk az eredményre **hibakorlátot**. Mivel az összes lehetséges hibaforrást figyelembe kell venni, ez nehéz feladat, ezért a teljes, szigorú hibakorlát meghatározása inkább csak numerikus programkönyvtárak rutinjainak vizsgálatakor szokásos.

## Hibabecslés differenciálható függvényeknél

Differenciálható függvény esetén az argumentum pontatlanságából adódó hiba a Lagrange-féle középtértéktételrel becsülhető. Mivel  $f(x) - f(\tilde{x}) = f'(\xi)(x - \tilde{x})$ , ahol  $\xi$  az  $x$  és  $\tilde{x}$  között van, adódik:

$$|f(x) - f(\tilde{x})| \leq |f'(\xi)| |x - \tilde{x}| \leq M_1 K_x$$

ahol  $M_1$  a derivált abszolút értékének,  $K_x$  pedig az  $x$  becslése hibájának egy felső korlátja.

*Példa:* ha az ismeretlen képletű, differenciálható  $f(x) = x^2$  függvény deriváltját numerikus differenciálással, az  $f'(x) \approx (f(x+h) - f(x)) / h$  képlettel becsüljük, akkor 1% mérési hibával, 4 értékes jegyet tárolva,  $h = 0,001$  lépésközzel az  $f'(1) = 2$  helyett  $(1,012 - 0,9900)/0,0010 = 22,00$  adódhat — a kis lépésköz és a mérési hiba együtt drámaian felnagyítja az eredmény hibáját.

## Összefoglalás — a hiba csökkentése

- Gondosan válasszuk meg a bemeneti adatokat és azok pontosságát (öröklött hiba).
- Lehetőleg **numerikusan stabil** algoritmust használjunk, és kerüljük a közel egyenlő számok kivonását (képlet- és kerekítési hiba).
- Ahol szükséges, alkalmazzunk nagyobb (kettős) pontosságot vagy intervallum-aritmetikát a hibák terjedésének korlátozására.

## 2. Számítógépes számábrázolás és kerekítés

A programozási nyelvek és a hardver a nagyobb hatékonyság kedvéért **véges, rögzített hosszúságú** számábrázolást alkalmaznak. Két leggyakoribb formátum a decimális egész ( $\pm\text{xxxxx}$ ) és a decimális lebegőpontos szám ( $\pm\text{xxxxx}\cdot 10^{\pm\text{xxx}}$ ). A véges ábrázolás az oka annak, hogy a gépi számítás eredménye rendszerint csak közelítés.

### Egész számok ábrázolása

Az egész számokat fix hosszúságú bitsorozat tárolja, az előjelet általában **kettes komplement** alakban kezelve (a legmagasabb helyiértékű bit az előjel). Az egész számok jellemzően 2 vagy 4 bájt hosszúak, így a tárolható legnagyobb (előjel nélküli) érték  $2^{15}$ , illetve  $2^{31}$  körüli.

Az egész számokkal általában kevesebb numerikus probléma van, de gondot okozhat a **túlcsordulás**: ha egy számláló értéke meghaladná a legnagyobb ábrázolható, „átfordulhat” negatívba (klasszikus példa a Tetris pontszáma). Sok feladatban épp ezért fontos, hogy csak egész értékek jöhessenek szóba — például banki műveletek számításakor.

### Lebegőpontos számok ábrázolása

A lebegőpontos szám az exponens előtti **mantisszából** és egy **exponensből** áll. Egy gyakori alak az, amelyben a mantissa első (tizedespont utáni) jegye nem nulla — ezt nevezzük **normalizált** alaknak. Kis abszolút értékű számoknak nincs mindig normalizált alakjuk (ilyen maga a nulla is); ezeket **denormalizáltak** hívjuk. Az általánosított normalizált lebegőpontos szám tehát egy  $L$  jegyű mantisszából és egy  $[-E, F]$  intervallumba eső exponensből áll, az alap (bázis) pedig rendszerint 2 hatványa:

$$x = (-1)^s \cdot m \cdot B^e, \quad B = 2$$

ahol  $s$  az előjel,  $m$  a mantissa,  $e$  az exponens. A PC-k az **IEEE 754** bináris lebegőpontos szabványt követik:

- **Szimpla pontosság** (real\*4): 24 bites mantissa, 8 bites exponens; a normalizált abszolút értékek kb.  $10^{-38}$  és  $10^{+38}$  között, ami kb. **7 értékes tizedesjegy**.
- **Dupla pontosság** (real\*8):  $L = 52$ ,  $E = 1022 = F - 1$ ; a normalizált abszolút értékek kb.  $10^{-308}$  és  $10^{+308}$  között, ami kb. **16 értékes tizedesjegy**.

A MATLAB belső ábrázolása dupla pontosságú, ezért a szimpla pontosság vele nem is illusztrálható; az egész, valós és komplex számok között bizonyos értelemben nem tesz különbséget. Megjegyzendő, hogy ezek a formátumok a *tárolt* számokra vonatkoznak — a processzoron belüli formátumok rendszerint pontosabbak.

### Kerekítés

A kerekítés arra szolgál, hogy a ténylegesen ábrázolható számok (a **gépi számok**) közül kiválasszuk azt, amellyel az eredményünket azonosítjuk.

- **Optimális kerekítés**: a számhoz legközelebbi gépi számot adja.
- **Levágás (csonkolás)**: egyszerűen elhagyja a jegyek egy részét (ha  $L$  jegyre kerekítünk, az  $L$ -edik utániakat). Könnyebb megvalósítani, de pontatlanabb.

Egy kerekítést **korrektnek** nevezünk, ha a szám és a kerekített értéke között nincs gépi szám; az optimális kerekítés és a levágás is korrekt.

*Példa:* a  $\pi = 3,14159265\dots$  négy jegyre vett optimális kerekítése 3,142, levágással kapott kerekítése pedig 3,141.

## Gépi pontosság (gépi epszilon)

A kerekítés relatív pontosságát a **gépi epszilon** jellemzi: a legkisebb olyan pozitív szám, amelyet 1-hez adva 1-nél nagyobb gépi számot kapunk. Korlátlan exponenshosszt és nem nulla értéket feltételezve a  $B$  bázisú,  $L$  jegyre való kerekítés relatív pontossága:

$$\varepsilon = B^{1-L} \qquad \varepsilon = \frac{1}{2} B^{1-L}$$

A bal oldali képlet a **korrekt**, a jobb oldali az **optimális** kerekítés relatív pontossága. Az állítás (pozitív, normalizált  $x = .x_1x_2\dots x_L\dots \cdot B^e$ ,  $x_1 \neq 0$  esetén) abból következik, hogy a korrekt kerekítés eltérése legfeljebb az  $L$ . pozíción vett egy egység, azaz  $B^{e-L}$ :

$$|x - \tilde{x}| \leq B^{e-L} = B^{e-1} B^{1-L} \leq |x| B^{1-L}.$$

## Felül- és alulcsordulás

Mivel az exponens hossza a gyakorlatban véges, az ábrázolandó szám lehet túl nagy (**felülcsordulás**) vagy túl kicsi (**alulcsordulás**) a normalizált ábrázoláshoz. Alulcsorduláskor denormalizált eredmény keletkezhet, ami ronthatja a relatív pontosságot; ezen a nagyságrendek megváltoztatása, az ún. **skálázás** segíthet. A kerekítési módok (köztük a kifelé kerekítés) az IEEE 754 szabvány részei.

### 3. Numerikus stabilitás, a hiba továbbterjedése, kondíciósám

Ez a tétel három, szorosan összefüggő fogalmat tárgyal: a numerikus stabilitást (mennyire „jól viselkedik” egy kiszámítási mód), a hiba továbbterjedését (hogyan nőnek a meglévő hibák), és a kondíciósámot (a feladat érzékenységének mérőszámát).

#### Numerikus stabilitás

A numerikus stabilitás **kvalitatív** fogalom: azt jelenti, hogy a függvény argumentumának kis megváltozása esetén a függvényértékben is csak kis, várható mértékű eltérés mutatkozik. Ennek ellentéte a numerikus **instabilitás**. A cél olyan eljárások létrehozása, amelyek a kiszámítandó függvények numerikusan **stabilis** változatát használják (ugyanazt az értéket másik, ekvivalens képlettel számolva).

Külön kell választani a **számábrázolás pontosságát** (precision) és egy **közelítő érték pontosságát** (accuracy). Jelölések:  $x \approx y$  (közelítőleg egyenlő),  $x \ll y$  (sokkal nagyobb).

Numerikus algoritmusokban kerülendő:

- **közel azonos számok kivonása** — a korábbi hibák felnagyítódnak, az eredmény relatív hibája megnő (jegyvesztés);
- **kis abszolút értékkel való osztás** vagy **nagy abszolút értékkel való szorzás** — ez az abszolút hiba növekedését okozza;
- általában minden olyan helyzet, amikor egy részsámítás eredménye lényegesen nagyobb vagy kisebb nagyságrendű, mint a végeredmény.

Néhány instabil kifejezés stabil átalakítása:

$$\sqrt{x+1} - \sqrt{x} = \frac{1}{\sqrt{x+1} + \sqrt{x}} \quad (x \gg 1)$$

$$1 - \cos x = \frac{\sin^2 x}{1 + \cos x} = 2 \sin^2 \frac{x}{2} \quad (x \approx 0)$$

A másodfokú egyenlet gyökeinél, ha  $b^2 \ll 4ac$ , a megszokott megoldóképlet jegyvesztést okoz; stabil helyette:

$$q = -\frac{1}{2} \left( b + \text{sign}(b) \sqrt{b^2 - 4ac} \right), \quad x_1 = \frac{q}{a}, \quad x_2 = \frac{c}{q}.$$

#### A hiba továbbterjedése és a kondíciósám

A numerikus számítás döntő kérdése, hogy a korábbi (esetleg öröklött) hibák milyen relatív hibához vezetnek. Bizonyos feladatoknál a relatív hibák növekedése **elkerülhetetlen** — ezeket **rosszul kondicionált** (ill-conditioned) feladatoknak nevezzük.

*Példa:* az  $f(x) = 1/(1-x)$  függvény az  $x = 0,999$  pont közelében; itt  $f(x) = 1000$ , és kis  $\varepsilon$  perturbációra  $f(x+\varepsilon) = 1000(1 + 10^3\varepsilon + 10^6\varepsilon^2 + \dots)$ . Az argumentum relatív hibája  $\approx \varepsilon$ , az eredményé viszont  $\approx 10^3\varepsilon$  — a kiszámítás módjától függetlenül a relatív hiba kb. ezerszeresére nő.

A rosszul kondicionáltság mérőszáma differenciálható kifejezésekre a **kondíciósám**,  $\kappa$ : a relatív hiba aszimptotikus növekedésének mértéke. Az  $\tilde{x} = (1+\varepsilon)x$  perturbáció Taylor-sorából:



$$K = \left| \frac{x f'(x)}{f(x)} \right|, \quad \frac{|f(x) - f(\tilde{x})|}{|f(x)|} \approx K \frac{|x - \tilde{x}|}{|x|}.$$

Tehát az eredmény relatív hibája az argumentum relatív hibájának kb.  $\kappa$ -szorososa. Eszerint  $\kappa < 1$  esetén **hibaelnyomásról**,  $\kappa > 1$  esetén a **hiba növeléséről** beszélünk; rosszul kondicionált feladatra  $\kappa \gg 1$ .

A függvények határértékbeli viselkedésének jellemzésére a „kis ordó” és a „nagy ordó” szolgál:  $f = o(g)$ , ha  $f/g \rightarrow 0$ ; és  $f = O(g)$ , ha  $f/g$  korlátos a vizsgált pont egy környezetében.

## Stabilitás és kondicionáltság — nem ugyanaz

A két fogalmat fontos megkülönböztetni. Tekintsük az  $f(x) = \sqrt{x^{-1}-1} - \sqrt{x^{-1}+1}$  függvényt. Az  $x = 0$  közelében  $x^{-1}-1 \approx x^{-1}+1$ , ezért a kifejezés **numerikusan instabil** (jegyvesztés), míg  $x = 1$  körül stabil. A kondíciószáma viszont

$$K = \frac{1}{2\sqrt{1-x^2}},$$

ami szerint a feladat az  $x = 0$  körül **jól** kondicionált ( $\kappa \approx 0,5$ ), az  $x = 1$  közelében viszont **rosszul** ( $\kappa \rightarrow \infty$ ). Más szóval a numerikus stabilitás (a képlet tulajdonsága) és a jó kondicionáltság (a feladat tulajdonsága) nem mindig jár együtt.

## 4. Intervallum aritmetika

Az intervallum-aritmetika a valós számokon végzett műveleteket terjeszti ki **intervallumokra**. Ha egy mennyiségről nem a pontos értékét, hanem csak azt tudjuk, hogy egy adott intervallumba esik, akkor az intervallumokon elvégzett művelet eredménye **garantáltan tartalmazza** az összes lehetséges valós eredményt. Ez teszi a módszert alkalmassá **bizonyító erejű** (verifikált) számításokra.

### Definíciók

**Halmazelméleti definíció:**  $A \square B := \{ a \square b : a \in A, b \in B \}$ , ahol  $A, B$  a valós kompakt intervallumok / halmazából valók (azaz  $[i, j]$  alakúak,  $i \leq j$ ).

**Aritmetikai definíció** (a négy alapl művelet):

$$[a, b] + [c, d] = [a + c, b + d]$$

$$[a, b] - [c, d] = [a - d, b - c]$$

$$[a, b] \cdot [c, d] = [\min(ac, ad, bc, bd), \max(ac, ad, bc, bd)]$$

$$[a, b]/[c, d] = [a, b] \cdot [1/d, 1/c] \quad (0 \notin [c, d])$$

Az osztásnál a  $0 \in [c, d]$  kikötés gyakori megszorításnak tűnik, de a tapasztalat szerint a gyakorlatban nem jelent komoly korlátot.

**Állítás (pontosság):** az aritmetikai definíció megfelel a halmazelméletinek és viszont — ebben az értelemben az intervallum-aritmetika **pontos**. A bizonyítás a szereplő műveletek monotonitásán múlik.

### Befoglaló függvény, természetes intervallum-kiterjesztés

Ha egy valós függvény műveleteit és standard függvényeit rendre az intervallumos megfelelőjükre cseréljük, az így kapott eljárást **természetes intervallum-kiterjesztésnek** nevezzük. Az  $F(X)$  az  $f(x)$  **befoglaló függvénye**, ha minden  $x \in X$ -re  $f(x) \in F(X)$ . A természetes kiterjesztés mindig befoglaló függvényt ad.

*Példa (túlbecslés):* az  $x - x^2$  valódi értékészlete a  $[0, 2]$  intervallumon  $[-2; 0,25]$ , a természetes intervallum-kiterjesztéssel kapott befoglalás viszont  $[-4, 2]$ . A befoglalás tehát helyes (tartalmazza a valódit), de tágabb a kelleténél — ez az ún. **függőségi (dependency) probléma**, amely a változó többszöri előfordulásából ered.

### Algebrai tulajdonságok

- $A +$  és  $a -$ , illetve  $a \cdot$  és  $a /$  **nem inverzei** egymásnak: pl.  $[0, 1] - [0, 1] = [-1, 1]$ , és  $[1, 2]/[1, 2] = [1/2, 2]$ .
- Érvényes a **szubdisztribúció**:  $A(B + C) \square AB + AC$  (valós  $a$  konstansra viszont egyenlőség:  $a(B+C) = aB + aC$ ).
- A  $0$  szélességű (pontoszerű) intervallumokra a műveletek megegyeznek a valós műveletekkel.
- Az összeadás és a szorzás kommutatív és asszociatív; az egyetlen egységelem az  $[1, 1]$ , az egyetlen zéruselem a  $[0, 0]$ .
- Érvényes a **befoglalási izotonitás**:  $A \square B, C \square D$  esetén  $A \square C \square B \square D$ .
- A szélesség  $w([a,b]) = b - a$ , a középpont  $m([a,b]) = (a+b)/2$ ; ezekre például  $w(aB) = |a| \cdot w(B)$  és  $m(A \pm B) = m(A) \pm m(B)$ .

## Gépi megvalósítás és alkalmazás

Gépi számokkal számolva ügyelni kell, hogy a kerekítés ne rontsa el a befoglalást: ehhez **kifelé kerekítést** kell alkalmazni (az alsó végpontot lefelé, a felsőt felfelé). Ezt az IEEE 754 szabvány támogatja, és több könyvtár teljes intervallum-kiterjesztést ad: **C-XSC**, **INTLAB**, **PROFIL/BIAS**.

Alkalmazás: intervallum-aritmetikára épülő korlátozás és szétválasztás (branch and bound) módszerrel a SIAM 2002-es 100 dolláros feladatainak egyikében a globális minimumra garantáltan helyes első 13 jegyet sikerült igazolni, mindössze 0,26 másodperc CPU-idő alatt — jól mutatva a verifikált számítás erejét.

## 5. Automatikus differenciálás

A numerikus algoritmusok gyakran támaszkodnak egy függvény deriváltjára. Az automatikus differenciálás (AD) olyan eljárás, amely a derivált **pontos** értékét (nem numerikus becslését) állítja elő közvetlenül a függvényt kiszámító programból.

### Motiváció — a derivált előállításának módjai

1. „**Kézi**” deriválás papíron, majd a megfelelő szubrutin megírása.
2. **Numerikus deriválás**: a differenciahányadossal,  $f'(x) \approx (f(x+h) - f(x))/h$ .
3. **Szimbolikus deriválás** számítógépes algebrai rendszerrel (Maple, Mathematica, Reduce, Derive, ...).
4. **Automatikus differenciálás**.

A **numerikus** derivált előnye (+) és hátránya (-): + nincs előzetes munka a deriváltak előállítására, + akkor is működik, ha csak a kiszámító szubrutin adott, a képlet nem; – a **levágási hiba** miatt sok értékes jegy veszik el, és – a gyorsan változó deriváltakra alkalmatlan. A **kézi** deriválás előnye, hogy nincs levágási hiba és a gyorsan változó deriváltak is jól meghatározhatók; hátránya, hogy munkaigényes, hibaforrás, és csak képlettel adott függvényre alkalmazható.

Az AD mindezek előnyeit egyesíti: nem igényel előzetes ráfordítást, akkor is működik, ha csak a kiszámító szubrutin adott, **nincs levágási hiba**, a gyorsan változó deriváltakra is alkalmas, és műveletigénye általában kisebb a numerikus deriválásénál.

### A módszer

Ha az  $f(x)$  függvény képlettel megadható, illetve van őt kiszámító szubrutin, a derivált értéke a következő eljárással kapható:

1. Minden változó helyett használjunk **kételemű adatszerkezetet** (érték, derivált): az első a megszokott változóérték, a második egy derivált-érték.
2. Minden **változóra** a második (derivált) komponens kezdetben **1**, minden **konstansra** pedig **0**.
3. Ezután minden műveletre két szabály kell: egy az első komponensre (a szokásos művelet) és egy a második komponensre (a deriválási szabály szerint).

*Példa:* ha  $f(x) = x_1 \cdot x_2$ , akkor a szokásos programsor  $F = X(1) \cdot X(2)$  helyett az AD a következőt számolja:

$$F(1) = X(1, 1) \cdot X(2, 1), \quad F(2) = X(1, 1) \cdot X(2, 2) + X(2, 1) \cdot X(1, 2).$$

Ez épp a szorzat deriválási szabálya — és sehol nem jelenik meg a deriváltfüggvény képlete.

### Elemi differenciálási szabályok

A (érték, derivált) pár második komponensét az alábbi szabályok szerint frissítjük ( $y = f(x)$ ):

- $a \pm x \rightarrow \pm 1$
- $a \cdot x \rightarrow a$
- $a / x \rightarrow -y/x$
- $\sqrt{x} \rightarrow 0,5/y$
- $\log(x) \rightarrow 1/x$
- $\exp(x) \rightarrow y$
- $\cos(x) \rightarrow -\sin(x)$

## Végrehajtási módok és műveletigény

Az AD-nek két végrehajtási módja van:

- **Sima (forward) változat:** az alapfüggvény kiszámítási sorrendjét követi, az argumentumoktól halad a függvényérték felé; **egymenetes**.
- **Fordított (reverse) módszer:** előbb meghatározza a kiszámítási fát, majd fordított sorrendben, a redundanciák kihasználásával számol; **két menetet** igényel és nagyobb a tárigénye, cserébe a teljes gradiens nagyon olcsón adódik.

A műveletigényt az  $L(\cdot)$  jelöli (az alapl műveletek  $\{+, -, \cdot, /, \sqrt{\cdot}, \log, \exp, \sin, \cos\}$  feletti költség). A gradiensre például a sima változat  $\leq 4nL(f)$ , a fordított pedig már  $\leq 4L(f)$  — utóbbi a változók  $n$  számától függetlenül. Ez teszi a fordított módot kulcsfontosságúvá sokváltozós feladatokban (és ez a mai gépi tanulás „visszaterjesztésének”, a backpropagationnek is az alapja).

## 6. Lineáris egyenletrendszerek megoldása Gauss eliminációval

A Gauss-elimináció **direkt** módszer lineáris egyenletrendszerek megoldására: véges sok lépésben (kerekítési hibától eltekintve) pontos eredményt ad. Lényege, hogy az  $Ax = b$  rendszert ekvivalens, **felső háromszög alakú** rendszerré alakítja, amelyből a megoldás visszahelyettesítéssel adódik.

### Az egyenletrendszer alakja

Mátrixos alakban  $Ax = b$ , ahol  $A$  az  $n \times n$ -es együtthatómátrix,  $x$  az ismeretlenek,  $b$  a jobb oldal oszlopvektora.

### Eliminációs mátrixok (Gauss-transzformációk)

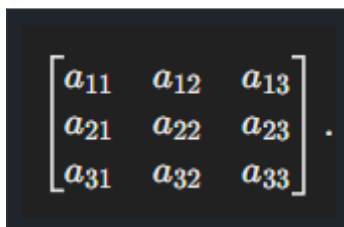
Az elimináció minden lépése egy **eliminációs mátrixszal** (Gauss-transzformációval) való balszorzásként írható le. Az  $M_k$  mátrix az aktuális oszlop főátló alatti elemeit nullázza:

$$M_k = I - m e_k^T, \quad m = \left(0, \dots, \frac{a_{k+1}}{a_k}, \dots, \frac{a_n}{a_k}\right)^T$$

ahol az  $a_k$  elemet **generálóelemnek** (főelemnek) nevezzük. Tulajdonságai:  $M_k$  alsó háromszögmátrix csupa 1-es főátlóval (tehát reguláris), és inverze pusztán a főátlón kívüli elemek előjelében tér el:  $M_k^{-1} = I + m e_k^T = L_k$ .

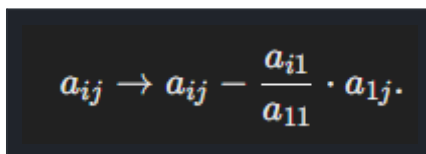
### A felső háromszög alak előállítása

Sorra alkalmazva az eliminációs mátrixokat felső háromszög alakot kapunk:

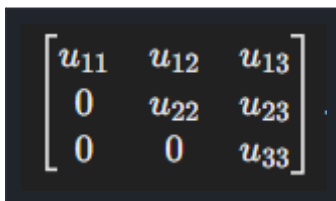

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}.$$

Kiindulási együtthatómátrix

A főátló alatti elemek nullázása oszloponként történik:


$$a_{ij} \rightarrow a_{ij} - \frac{a_{i1}}{a_{11}} \cdot a_{1j}.$$

Az első oszlop nullázása ( $a_{21}$ ,  $a_{31}$ )


$$\begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix}.$$

A kapott felső háromszög alak

A háromszög alakból a megoldás **visszahelyettesítéssel**, az utolsó egyenlettől visszafelé adódik:

$$x_n = \frac{b_n}{u_{nn}}.$$

*Az utolsó ismeretlen*

$$x_{n-1} = \frac{b_{n-1} - \sum_{j=n}^{n-1} u_{n-1,j} \cdot x_j}{u_{n-1,n-1}}.$$

*A többi ismeretlen visszahelyettesítéssel*

## Pivotálás (főelemkiválasztás) és stabilitás

A vezető elem ( $a_{ii}$ ) megválasztása kritikus: ha közel van nullához, az algoritmus numerikusan instabil. Ezért a hibaforrásoknál mondottak alapján **nagy abszolút értékű** főelemet érdemes választani.

- **Részleges főelemkiválasztás:** az aktuális oszlop legnagyobb abszolút értékű elemét hozzuk a főátlóba sorcserével.
- **Teljes főelemkiválasztás:** a még szóba jövő teljes részmátrixból választjuk a főelemet (sor- és oszlopcsere). Bár a permutáció „elrontja” az  $L$  és  $U$  alakot, a megoldás szempontjából ezek teljes értékűek maradnak.

## Kidolgozott példa

Tekintsük a következő egyenletrendszert:

$$\begin{aligned} 2x + 4y + z &= 10, \\ 4x + 11y + z &= 24, \\ 6x + 17y + z &= 35. \end{aligned}$$

*A megoldandó egyenletrendszer*

Mátrix alakban:

$$A = \begin{bmatrix} 2 & 4 & 1 \\ 4 & 11 & 1 \\ 6 & 17 & 1 \end{bmatrix}, \quad b = \begin{bmatrix} 10 \\ 24 \\ 35 \end{bmatrix}.$$

*Mátrix alak*

Az első oszlop nullázása:

$$R_2 \rightarrow R_2 - 2R_1, \quad R_3 \rightarrow R_3 - 3R_1.$$

*Első oszlop eliminációja*

$$A = \begin{bmatrix} 2 & 4 & 1 \\ 0 & 3 & -1 \\ 0 & 5 & -2 \end{bmatrix}, \quad b = \begin{bmatrix} 10 \\ 4 \\ 5 \end{bmatrix}.$$

*Az eredmény*

A második oszlop nullázása:

$$R_3 \rightarrow R_3 - \frac{5}{3}R_2.$$

*Második oszlop eliminációja*

$$A = \begin{bmatrix} 2 & 4 & 1 \\ 0 & 3 & -1 \\ 0 & 0 & -\frac{1}{3} \end{bmatrix}, \quad b = \begin{bmatrix} 10 \\ 4 \\ \frac{1}{3} \end{bmatrix}.$$

*Felső háromszög alak*

Visszahelyettesítés és a megoldás:

$$z = \frac{\frac{1}{3}}{-\frac{1}{3}} = -1, \quad y = \frac{4 - (-1)}{3} = 1.67, \quad x = \frac{10 - 4 \cdot 1.67 - (-1)}{2} = 0.33.$$

*Visszahelyettesítés*

$$x = 0.33, y = 1.67, z = -1.$$

*A megoldás*

## Műveletigény, előnyök és hátrányok

Az elimináció (LU felbontás) műveletigénye közel  $n^3/3$  szorzás és összeadás, a visszahelyettesítés további  $\sim n^2$ .

- **Előny:** direkt módszer, kis és közepes méretű (sűrű) rendszerekre hatékony és megbízható.
- **Hátrány:** pivotálás nélkül instabil lehet; nagy, ritka mátrixoknál az elimináció „feltöltődést” okozhat (a mátrix sűrűbbé válik), ezért ott iterációs módszerek versenyképesebbek.



## 7. LU felbontás, eliminációs mátrixok, főelemkiválasztás

Az LU felbontás a Gauss-elimináció „mátrixos” megfogalmazása: az  $A$  mátrixot egy alsó ( $L$ ) és egy felső ( $U$ ) háromszögmátrix szorzatára bontja.

### LU felbontás

$$A = LU,$$

$$A = LU$$

ahol  $L$  alsó háromszögmátrix csupa 1-es főátlóval,  $U$  felső háromszögmátrix. A felbontás abból adódik, hogy az eliminációs mátrixok szorzata háromszög alakra hozza  $A$ -t:

$$M_{n-1} \cdots M_1 A = U, \quad A = LU, \quad L = M_1^{-1} \cdots M_{n-1}^{-1} = L_1 \cdots L_{n-1}.$$

Ennek nagy előnye, hogy ha több jobb oldalra kell megoldani a rendszert, a felbontást csak egyszer kell elvégezni; utána minden rendszer két háromszög-rendszerre esik szét:

- $Ly = b$  (előrehelyettesítés), majd
- $Ux = y$  (visszahelyettesítés).

### Az elimináció lépései

Kezdetben  $L = I$  és  $U = A$ . Minden  $k$  oszlopra az  $L$ -be az eliminációs tényező kerül,  $U$ -ból pedig kivonjuk az eliminált sort:

$$L_{ji} = \frac{U_{ji}}{U_{ii}} \quad (j > i).$$

Az eliminációs tényező ( $L$  eleme)

$$U_{jk} = U_{jk} - L_{ji} \cdot U_{ik}.$$

$U$  sorának módosítása

*Példa.* A felbontandó mátrix:

$$A = \begin{bmatrix} 2 & 3 & 1 \\ 4 & 7 & 3 \\ 6 & 18 & 5 \end{bmatrix}$$

$A$  kiindulási mátrix

$$L_{21} = \frac{4}{2} = 2, \quad L_{31} = \frac{6}{2} = 3.$$

Első oszlop eliminációja

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 3 & 0 & 1 \end{bmatrix}.$$

Az  $L$  mátrix

$$U = \begin{bmatrix} 2 & 3 & 1 \\ 0 & 1 & 1 \\ 0 & 9 & 2 \end{bmatrix}.$$

Az  $U$  módosítása

$$L_{32} = \frac{9}{1} = 9.$$

Második oszlop eliminációja

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 3 & 9 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 2 & 3 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & -7 \end{bmatrix}.$$

Az  $L$  és  $U$  végleges alakja

## Eliminációs mátrixok

Az eliminációs mátrixok (Gauss-transzformációk) az egyes sortranszformációkat reprezentálják; szorzatuk az egész eliminációt elvégzi:

$$E_k \cdot E_{k-1} \cdots E_1 \cdot A = U.$$

Az eliminációk szorzata

$$E_1 = \begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ -3 & 0 & 1 \end{bmatrix}.$$

Egy  $E_1$  eliminációs mátrix

Fontos azonosság ( $k < j$  esetén):  $M_k M_j = I - m e_k^T - t e_j^T$ , az inverzekre pedig  $L_k L_j = I + m e_k^T + t e_j^T$  — vagyis az  $L$  egyszerűen „összegyűjti” az eliminációs tényezőket.

## Főelemkiválasztás (pivotálás)

A pivotálás célja a numerikus stabilitás: kis vagy nulla közeli főelemek instabilitást okoznak. Szükség is lehet rá — például az  $A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$  szinguláris mátrixnál a főátlóbeli elem nulla. A **részleges** főelemkiválasztás csak sorcserét használ (az oszlop legnagyobb abszolút értékű elemét hozza a főátlóba), a **teljes** főelemkiválasztás sor- és oszlopcserét is.

$$\begin{bmatrix} 0 & 2 & 1 \\ 1 & 3 & 2 \\ 2 & 4 & 3 \end{bmatrix}.$$

*A mátrix pivotálás előtt*

$$\begin{bmatrix} 2 & 4 & 3 \\ 1 & 3 & 2 \\ 0 & 2 & 1 \end{bmatrix}.$$

*Részleges pivotálás után*

A MATLAB `lu` rutinja a részleges főelemkiválasztást kezeli. Az  $[L,U,P]=lu(A)$  az  $L$ ,  $U$  mellett a  $P$  permutációs mátrixot is megadja ( $PA = LU$ ); két kimenettel az  $L$  permutált alakban jön. A felbontás műveletigénye közel  $n^3/3$ .

## Alkalmazások

- **Egyenletrendszer:** az  $Ly = b$ ,  $Ux = y$  lépésekkel gyorsan megoldható.
- **Inverz:** az  $Ax_i = e_i$  rendszerek megoldásával oszloponként.
- **Determináns:** az  $U$  főátlóelemeinek szorzata (pivotálás esetén az előjelcserék figyelembevételével):

$$\det(A) = \prod_{i=1}^n U_{ii}.$$

*Determináns az  $U$  főátlójából*

## 8. Mátrixinvertálás, mátrixnormák, mátrixok kondíciószáma

### Mátrixinvertálás

Egy négyzetes  $A$  mátrix inverze az  $A^{-1}$ , amelyre:

$$A \cdot A^{-1} = I,$$

Az inverz definíciója:  $A \cdot A^{-1} = I$

ahol  $I$  az egységmátrix. A mátrix akkor invertálható (reguláris), ha  $\det(A) \neq 0$ . Számítási módszerek:

- **Gauss-elimináció alapuló:** oldjuk meg az  $Ax_i = e_i$  rendszereket ( $e_i$  az egységmátrix oszlopai). Az  $MA = U$  alakra támaszkodva  $n$  darab felső háromszög-rendszert ( $Ux_i = Me_i$ ) kell megoldani; az  $x_i$  oszlopok adják  $A^{-1}$ -et. (Ez a Gauss–Jordan, illetve LU alapú eljárás.)
- **Adjungált módszer** (kis méretre):

$$A^{-1} = \frac{\text{adj}(A)}{\det(A)},$$

Inverz az adjungálttal:  $A^{-1} = \text{adj}(A)/\det(A)$

*Példa.*

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}.$$

*A mátrix*

$$\det(A) = (1 \cdot 4) - (2 \cdot 3) = -2.$$

*A determináns*

$$\text{adj}(A) = \begin{bmatrix} 4 & -2 \\ -3 & 1 \end{bmatrix}.$$

*Az adjungált mátrix*

$$A^{-1} = \frac{1}{-2} \begin{bmatrix} 4 & -2 \\ -3 & 1 \end{bmatrix} = \begin{bmatrix} -2 & 1 \\ 1.5 & -0.5 \end{bmatrix}$$

*Az inverz*

A MATLAB-ban az `inv(A)` adja az inverzet. Ha az nem létezik, „Matrix is singular to working precision” figyelmeztetést és Inf elemeket kapunk. A gyakorlatban egyenletrendszer megoldására ritkán számítjuk ki explicit az inverzet — a backslash (`A\b`) numerikusan stabilabb és olcsóbb.

## Mátrixnormák

A  $\|\cdot\| : \mathbb{R}^{n \times n} \rightarrow \mathbb{R}$  leképezés **mátrixnorma**, ha: (i)  $\|A\| > 0$ , ha  $A \neq 0$ ; (ii)  $\|\alpha A\| = |\alpha| \cdot \|A\|$ ; (iii)  $\|A+B\| \leq \|A\| + \|B\|$ ; (iv)  $\|AB\| \leq \|A\| \cdot \|B\|$ . Minden vektornormához tartozik egy **indukált** mátrixnorma:

$$\|A\| := \sup_{x \neq 0} \frac{\|Ax\|}{\|x\|} = \sup_{\|x\|=1} \|Ax\|.$$

A kis műveletigényű, könnyen számolható normák:

$$\|A\|_F = \sqrt{\sum_{i,j} |a_{ij}|^2}.$$

Frobenius-norma

$$\|A\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^n |a_{ij}|,$$

1-norma (oszlopösszeg-maximum)

$$\|A\|_\infty = \max_{1 \leq i \leq n} \sum_{j=1}^n |a_{ij}|,$$

$\infty$ -norma (sorösszeg-maximum)

$$\|A\|_2 = \sqrt{\lambda_{\max}(A^T A)},$$

Spektrális (2-)norma

A  $\|A\|_1$  az oszlopok abszolútérték-összegének maximuma, a  $\|A\|_\infty$  a soroké, a Frobenius-norma az elemek négyzetösszegének gyöke, a spektrális norma pedig az  $A^T A$  legnagyobb sajátértékének gyöke. (A MATLAB `norm(A)` alapból a 2-normát adja.)

## Mátrixok kondíciószáma

A kondíciós szám azt méri, mennyire érzékeny a megoldás a bemeneti hibákra:

$$\text{cond}(A) = \|A\| \cdot \|A^{-1}\|.$$

Szinguláris mátrixra végtelennek definiáljuk; függ a normától ( $\text{cond}_p$ ), de a jól/rosszul kondicionáltság maga jórészt normafüggetlen. Főbb tulajdonságok:

- $\text{cond}(I) = 1$ , és általában  $\text{cond}(A) \geq 1$ ;
- $\text{cond}(A) = \text{cond}(A^{-1})$ , és  $\text{cond}(cA) = \text{cond}(A)$  ( $c \neq 0$ );
- permutációs mátrixra  $\text{cond}(P) = 1$ , diagonálisra  $\text{cond}(D) = \max |d_{ii}| / \min |d_{ii}|$ ;
- $\text{cond}(AB) \leq \text{cond}(A) \cdot \text{cond}(B)$ , és  $\text{cond}_\infty(A) = \text{cond}_1(A^T)$ .

*Példa.* A kondíciós szám nem a determinánssal mérhető: a  $0,1 \cdot I$  mátrix determinánsa  $10^{-n}$  (kicsi), mégis  $\text{cond}(0,1 \cdot I) = 1$ . Ezzel szemben az egységmátrixtól a főátló feletti  $-1$ -esekben eltérő mátrixra  $\det = 1$ , de  $\text{cond}_1 = n \cdot 2^{n-1}$  — nagy  $n$ -re csaknem szinguláris.

$$\kappa(A) = \|A\| \cdot \|A^{-1}\|.$$

*A kondíciós szám definíciója*

Számolt példa:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}.$$

*A mátrix*

$$\|A\|_1 = \max(|1+3|, |2+4|) = 6.$$

*Az 1-norma*

$$\|A^{-1}\|_1 = \max(|-2+1.5|, |1+0.5|) = 3.5.$$

*Az inverz 1-normája*

$$\kappa(A) = \|A\|_1 \cdot \|A^{-1}\|_1 = 6 \cdot 3.5 = 21.$$

*A kondíciós szám*

## Kondíciós szám és stabilitás

Jól kondicionált feladatnál ( $\text{cond} \approx 1$ ) a bemeneti hiba alig nagyítódik; rosszul kondicionáltnál ( $\text{cond} \gg 1$ ) erősen felerősödik, és a rendszer közel szinguláris:

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 1.0001 \end{bmatrix}.$$

*Rosszul kondicionált mátrix*

A kondíciós szám így alapvető a hibabecslésben és a numerikus algoritmusok érzékenységének vizsgálatában.

## 9. Cholesky felbontás, QR ortogonális felbontás

Lineáris egyenletrendszereknél érdemes kihasználni a mátrix szerkezetét. Külön, hatékonyabb módszer jár a **szimmetrikus**, **pozitív definit**, a **sáv**- és a **ritka** mátrixoknak. Az első esetre a Cholesky-felbontás való, általános mátrixokra pedig a QR-felbontás kínál stabil alternatívát.

### Cholesky-felbontás

Ha  $A$  szimmetrikus és pozitív definit, akkor az LU-felbontása  $U = L^T$  alakban létezik, azaz

$$A = LL^T,$$

$$A = L \cdot L^T$$

ahol  $L$  alsó háromszögmátrix **pozitív** főátlóelemekkel (nem feltétlen 1-esek). Az  $L$  elemei közvetlenül adódnak:

$$L_{ii} = \sqrt{A_{ii} - \sum_{k=1}^{i-1} L_{ik}^2}.$$

*A főátló elemei*

$$L_{ij} = \frac{A_{ij} - \sum_{k=1}^{j-1} L_{ik}L_{jk}}{L_{jj}}, \quad (i > j).$$

*Az alsó háromszög többi eleme*

A felbontás után a rendszer két lépésben oldható:  $Ly = b$ , majd  $L^Tx = y$ .

*Példa.*

$$A = \begin{bmatrix} 4 & 2 & 4 \\ 2 & 10 & 5 \\ 4 & 5 & 12 \end{bmatrix}.$$

*A felbontandó mátrix*

$$L = \begin{bmatrix} 2 & 0 & 0 \\ 1 & 3 & 0 \\ 2 & 1 & 2 \end{bmatrix},$$

*Az  $L$  tényező*

- $L_{11} = \sqrt{4} = 2,$
- $L_{21} = \frac{2}{2} = 1,$
- $L_{22} = \sqrt{10 - 1^2} = 3, \text{ stb.}$

Ellenőrzés

A Cholesky-felbontás (ejtsd: „koleszki”) numerikusan stabil, műveletigénye  $\approx n^3/3$  — feleannyi, mint egy általános LU-felbontásé. MATLAB-ban a  $\text{chol}(A)$  az  $R = L^T$  felső háromszöget adja ( $R^T R = A$ ); a megoldás:  $y = R \backslash b$ ;  $x = R \backslash y$ .

## QR ortogonális felbontás

A QR-felbontás egy  $A \in \mathbb{R}^{m \times n}$  mátrixot egy **ortogonális**  $Q$  (azaz  $Q^T Q = I$ ) és egy felső háromszög  $R$  szorzatára bont:

$$A = QR,$$

$$A = QR$$

- $Q \in \mathbb{R}^{m \times m}$  egy ortogonális mátrix ( $Q^T Q = I$ ),
- $R \in \mathbb{R}^{m \times n}$  egy felső háromszögmátrix.

$A$   $Q$  és  $R$  tulajdonságai

Ortogonalitása miatt a QR numerikusan stabilabb az LU-nál, és nem igényel szimmetriát vagy pozitív definitiséget. Előállítási módjai:

- **Gram–Schmidt-ortogonalizáció:** az oszlopokat ortonormált bázissá alakítja.

$$q_1 = \frac{a_1}{\|a_1\|},$$

Az első oszlop

$$q_k = a_k - \sum_{i=1}^{k-1} \text{proj}_{q_i}(a_k), \quad \text{ahol} \quad \text{proj}_{q_i}(a_k) = \frac{q_i^T a_k}{q_i^T q_i} q_i.$$

A további oszlopok ortogonalizálása

- **Givens-rotáció:** elemenkénti nullázás forgatásokkal (ritka mátrixokra hatékony).
- **Householder-reflexió:** a  $H = I - 2vv^T$  tükrözéssel egész oszloprészeket nulláz; ez a leggyakoribb, numerikusan kedvező eljárás.

*Példa (Gram–Schmidt):*

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}.$$



A mátrix

$$Q = \begin{bmatrix} 1/\sqrt{3} & 1/\sqrt{2} \\ 1/\sqrt{3} & 0 \\ 1/\sqrt{3} & -1/\sqrt{2} \end{bmatrix}, \quad R = \begin{bmatrix} \sqrt{3} & 3.74 \\ 0 & 0.83 \end{bmatrix}.$$

A felbontás eredménye

## Alkalmazások

- **Egyenletrendszer:**  $Ax = b$  esetén  $QRx = b$ , innen

$$Q^T b = Rx,$$

$$Rx = Q^T b$$

majd  $Rx = Q^T b$  visszahelyettesítéssel.

- **Legkisebb négyzetek:** az  $A^T Ax = A^T b$  normálegyenlet QR-rel stabilan kezelhető (a hibás kondíciójú  $A^T A$  explicit képzése nélkül).
- **Sajátértékek:** az iteratív QR-algoritmus a sajátértékek meghatározásának alapmódszere.

## Összefoglalás

A Cholesky gyors és stabil választás szimmetrikus pozitív definit mátrixokra (fele annyi művelet), a QR pedig általánosan alkalmazható, ortogonalitása miatt jól kondicionált eljárás — különösen túlhatározott (legkisebb négyzetes) feladatokra és sajátérték-számításra.

## 10. Mátrixok sajátértékei, sajátvektorai, a sajátértékek korlátai, a hatványmódszer

### Sajátérték és sajátvektor

Egy  $A$  négyzetes mátrixhoz keressük azt a  $\lambda$  skalárt és  $x \neq 0$  vektort, amelyre

$$Av = \lambda v,$$

*A sajátérték-egyenlet:  $Ax = \lambda x$*

Ekkor  $\lambda$  az  $A$  **sajátértéke**,  $x$  a hozzá tartozó **sajátvektora**. Hasonlóan értelmezhető a *baloldali* sajátvektor:  $y^T A = \lambda y^T$  (ez  $A^T$  sajátvektora). A sajátértékek halmaza a mátrix **spektruma**,  $\lambda(A)$ ; a legnagyobb abszolút értékű sajátérték a **spektrálsugár**,  $\rho(A) = \max\{|\lambda| : \lambda \in \lambda(A)\}$ .

A sajátvektorok nem egyértelműek (skalárszorosuk is sajátvektor), komplex esetben pedig az  $A^T$  helyett az  $A^H$  konjugált transzponáltat használjuk. A sajátértékeket az  $(A - \lambda I)x = 0$  homogén rendszer nemtriviális megoldhatóságából, azaz a **karakterisztikus egyenletből** kapjuk:

$$\det(A - \lambda I) = 0,$$

*$\det(A - \lambda I) = 0$*

*Példa.*

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}.$$

*A mátrix*

$$\det \left( \begin{bmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{bmatrix} \right) = (2-\lambda)^2 - 1 = 0.$$

*A karakterisztikus egyenlet*

$$\lambda_1 = 3, \quad \lambda_2 = 1.$$

*A sajátértékek*

A sajátvektorok az  $(A - \lambda I)x = 0$  rendszer megoldásai:

$$(A - \lambda I)v = 0$$

*A sajátvektorok homogén rendszere*

### A sajátértékek korlátai

**Tétel.** Tetszőleges  $A$  mátrixra és bármely (vektornormából származó) mátrixnormára a spektrálsugár nem nagyobb a normánál:  $\rho(A) \leq \|A\|$ . Így a könnyen számolható 1-,  $\infty$ - és Frobenius-norma egyaránt felső korlátot ad.

$$(\|A\|_2).$$

*Spektrális korlát*

$$\sqrt{\sum_{i=1}^n \lambda_i^2} \leq \|A\|_F.$$

*Frobenius-korlát*

$$\min_i \|A_{i,\cdot}\|_\infty \leq \rho(A) \leq \max_i \|A_{i,\cdot}\|_\infty.$$

*Sor- és oszlopösszeg-korlátok*

Élesebb lokalizációt ad a **Gersgorin-tétel**: minden sajátérték benne van a komplex sík alábbi körei egyesítésében:

$$|z - a_{ii}| \leq \sum_{k \neq i} |a_{ik}|, \quad i = 1, \dots, n.$$

Ráadásul, ha a körök egy  $m$  és egy  $n-m$  elemű, diszjunkt csoportra bomlanak, akkor az egyik csoport pontosan  $m$  sajátértéket tartalmaz (multiplicitással).

## A hatványmódszer

A hatványmódszer (von Mises-féle vektoriteráció) a **legnagyobb** abszolút értékű sajátértéket és sajátvektorát közelíti az

$$y^k = Ax^k, \quad x^{k+1} = \frac{y^k}{\|y^k\|}$$

iterációval. A  $x^0 \neq 0$  kezdővektor nem lehet merőleges a keresett sajátvektorra. Ha  $|\lambda_1| > |\lambda_i|$  ( $i \geq 2$ ), akkor  $\text{sign}(\lambda_1)^k x^k \rightarrow u_1$  és  $\|y^k\|/\|x^k\| \rightarrow |\lambda_1|$ .

$$v_{k+1} = Av_k, \quad v_{k+1} = \frac{v_{k+1}}{\|v_{k+1}\|}.$$

*Az iteráció*

$$\lambda_{\max} \approx \frac{v_k^T Av_k}{v_k^T v_k}.$$

*$\lambda_{\max}$  becslése*

A becslés jobb a **Rayleigh-hányadossal**, amelynek konvergenciája gyorsabb:

$$\frac{(x^k)^T y^k}{(x^k)^T x^k} = \lambda_1 + O\left(\left(\frac{\lambda_2}{\lambda_1}\right)^{2k}\right).$$

*Példa.*

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}, \quad v_0 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

*A mátrix*

$$1. \quad v_1 = Av_0 = \begin{bmatrix} 3 \\ 3 \end{bmatrix}, \text{ normálás: } v_1 = \begin{bmatrix} 0.707 \\ 0.707 \end{bmatrix}.$$

*Az iteráció lépései*

$$v_2 = Av_1 = \begin{bmatrix} 2.121 \\ 2.121 \end{bmatrix}, \text{ normálás: } v_2 = \begin{bmatrix} 0.707 \\ 0.707 \end{bmatrix}.$$

*A konvergencia*

## Változatok

- **Inverz hatványmódszer** (Wielandt-féle inverz iteráció): mivel  $A^{-1}$  sajátértéke  $1/\lambda$  ugyanazzal a sajátvektorral, az  $Ay = x^k$ ,  $x^{k+1} = y/\|y\|$  iteráció a **legkisebb** abszolút értékű sajátértéket adja.
- **Eltolás (shift)**: az  $A - \sigma I$  mátrixra alkalmazva a  $\sigma$ -tól **legtávolabbi** sajátértéket kapjuk; alkalmas  $\sigma$  választással bármelyik sajátérték megcélozható.

A sajátértékek és sajátvektorok kulcsszerepet játszanak a stabilitásvizsgálatban, a dimenziócsökkentésben (PCA) és a spektrális gráfelméletben.

# 11. A mátrixok típusai, hasonlósági transzformáció

## Mátrixtípusok

A sajátérték-számító algoritmus megválasztását erősen befolyásolja a mátrix szerkezete (valós vagy komplex; kicsi és sűrű vagy nagy és ritka; szimmetrikus-e). A legfontosabb típusok:

- **Négyzetes:** a sorok és oszlopok száma megegyezik.

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}.$$

*Négyzetes mátrix*

- **Diagonális:** csak a főátlón van nemnulla elem (könnyen invertálható).

$$D = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix}.$$

*Diagonális mátrix*

- **Szimmetrikus:**  $A = A^T$ .

$$A = A^T.$$

$A = A^T$

$$A = \begin{bmatrix} 1 & 2 \\ 2 & 3 \end{bmatrix}.$$

*Szimmetrikus példa*

- **Ortogonalis:**  $A^T A = I$  (megőrzi a hosszat és a szöget).

$$Q = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}.$$

*Ortogonalis példa*

- **Felső/alsó háromszög:** csak a főátló felett, illetve alatt van nemnulla elem.

$$U = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 4 & 5 \\ 0 & 0 & 6 \end{bmatrix}$$

*Felső háromszög*

Két további, az elméletben fontos típus: **pozitív definit** (szimmetrikus, és  $x^T A x > 0$  minden  $x \neq 0$ -ra) és **normális** ( $AA^T = A^T A$ ). Komplex esetben a transzponálás helyébe a konjugált transzponálás lép, és a megfelelő nevek: **Hermite-szimmetrikus** (hermitikus), **unitér** és normális. (A speciálisabb idempotens

—  $A^2 = A$  — és nilpotens —  $A^k = 0$  — típusok is előfordulnak.)

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}.$$

*Idempotens példa*

$$A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \quad A^2 = 0.$$

*Nilpotens példa*

## Hasonlósági transzformáció

Az  $A$  mátrix **hasonló**  $B$ -hez, ha van olyan reguláris  $T$ , amelyre

$$B = P^{-1}AP.$$

$$B = T^{-1}AT$$

A hasonlóság megőrzi a spektrumot:  $By = \lambda y$  esetén  $A(Ty) = \lambda(Ty)$ , tehát  $\lambda$  az  $A$ -nak is sajátértéke (más,  $Ty$  sajátvektorral). Megőrzi a determinánst és a nyomot is:

$$\det(A) = \det(B).$$

*A determináns megőrzése*

$$\operatorname{tr}(A) = \operatorname{tr}(B).$$

*A nyom megőrzése*

Vigyázat: a fordított irány nem igaz — az azonos sajátértékű mátrixok nem feltétlenül hasonlóak (pl. az egység mátrix és az  $\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$  mátrix nem hasonló).

## Diagonalizálás, Jordan- és Schur-alak

Ha az  $A$  sajátvektoraiból összeállított  $X$  reguláris, akkor  $A = XDX^{-1}$ , ahol  $D$  a sajátértékek diagonális mátrixa:

$$A = PDP^{-1}.$$

$$A = PDP^{-1}$$

**Tétel.** Az  $A$  pontosan akkor diagonalizálható, ha van  $n$  lineárisan független sajátvektora. Ha nincs, általában csak **Jordan-alakra** hozható (a felső mellékátlóban lehetnek nemnulla elemek) — ez azonban numerikusan instabil, ezért a MATLAB-ban nincs is jordan rutin. Stabil viszont a **Schur-féle normálalak** (felső háromszög, unitér  $T$ -vel), amelynek főátlójában szintén a sajátértékek állnak.

A típushoz tartozó hasonlósági transzformáció és eredmény:

- páronként eltérő sajátértékek  $\rightarrow$  reguláris  $T \rightarrow$  **diagonális**;
- szimmetrikus  $\rightarrow$  ortogonális  $\rightarrow$  valós diagonális;

- Hermite-szimmetrikus  $\rightarrow$  unitér  $\rightarrow$  valós diagonális;
- normális  $\rightarrow$  unitér  $\rightarrow$  diagonális;
- tetszőleges  $\rightarrow$  unitér  $\rightarrow$  **felső háromszög (Schur)**;
- tetszőleges  $\rightarrow$  reguláris  $\rightarrow$  **Jordan-alak**.

*Példa (diagonalizálás):*

$$A = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix}.$$

*A mátrix*

$$\lambda_1 = 5, \lambda_2 = 2.$$

*A sajátértékek*

$$P = \begin{bmatrix} 1 & -1 \\ 2 & 1 \end{bmatrix}, \quad D = \begin{bmatrix} 5 & 0 \\ 0 & 2 \end{bmatrix}.$$

*A diagonalizálás*

MATLAB: eig (összes sajátérték/-vektor), eigs (néhány, ritka mátrixra), hess (Hessenberg-alak), schur (Schur-alak), poly (karakterisztikus polinom), condeig (sajátértékek kondíciószáma).

## 12. A sajátértékek és a sajátvektorok kondicionáltsága

A kondicionáltság azt méri, hogy a mátrix együtthatóinak **kis változása** mennyire befolyásolja a sajátértékeket és a sajátvektorokat. Ez más kérdés, mint a vele felírt egyenletrendszer megoldásának érzékenysége — egyazon mátrixon belül is lehetnek eltérő érzékenyséű sajátértékek.

### A sajátértékek kondicionáltsága

Egy  $A$  mátrix perturbációja ( $\Delta A$ ) a  $\lambda$  sajátértéket első rendben a jobb ( $x$ ) és bal ( $y$ ) sajátvektorokon keresztül változtatja:

$$|\Delta \lambda| \leq \|\Delta A\| \cdot \|v\| \cdot \|u\|,$$

*A sajátérték perturbációja*

A  $\lambda$  sajátérték **kondíciószáma** a normalizált sajátvektorokkal ( $x^H x = y^H y = 1$ ):

$$\kappa(\lambda) = \frac{1}{|y^H x|}.$$

$$\kappa(\lambda) = \frac{|u^T v|}{\|u\| \cdot \|v\|}.$$

*A kondíciószám képlete*

Ez épp a bal és jobb sajátvektor által bezárt szög koszinuszának reciproka: ha ez nagy (a két vektor közel merőleges),  $\lambda$  **rosszul** kondicionált.

Fontos következmény: **szimmetrikus** (Hermite-szimmetrikus) mátrixnál a bal és jobb sajátvektorok megegyeznek, így  $y^H x = 1$ , és  $\kappa(\lambda) = 1$  — a sajátértékek mindig jól kondicionáltak. Általánosabban a **normális** mátrixok sajátértékei is jól kondicionáltak.

*Példa.* Egy felső háromszög mátrixnál ( $\lambda_1 = 1$ ,  $\lambda_2 = 0,999$ ,  $\lambda_3 = 2$ ) az első két sajátérték kondíciószáma  $\sim 2,5 \cdot 10^3$ , a harmadiké csak  $\sim 5,2$ . Az  $a_{31}$  elem  $10^{-6}$ -os megváltoztatása a  $\lambda_1$ ,  $\lambda_2$  párt komplexszé teszi ( $\approx 0,9995 \pm 0,0013i$ ), míg  $\lambda_3 = 2$  változatlan marad — szemléltetve a rossz kondicionáltságot. (MATLAB: condeig.)

### A sajátvektorok kondicionáltsága

A sajátvektorok érzékenységet a hozzájuk tartozó sajátérték kondíciója és a **többi sajátértéktől mért távolsága** (a spektrális rés, gap) határozza meg:

$$(\text{gap} = |\lambda_i - \lambda_j|)$$

*A spektrális rés*

- **Nagy** rés (jól elkülönülő sajátértékek)  $\rightarrow$  stabil sajátvektorok.
- **Kis** rés (közeli vagy többszörös sajátértékek)  $\rightarrow$  instabil sajátvektorok.

*Példa.* A  $\text{diag}(1; 0,99; 2)$  mátrix szimmetrikus, sajátértékei jól kondicionáltak; ám az  $a_{12} = a_{21} = 10^{-4}$  kis perturbáció az első két sajátvektort jelentősen elforgatja (a harmadik változatlan), épp mert a megfelelő



sajátértékek (1 és 0,99) közel vannak.

## A kondícionáltság javítása

- **Skálázás / kiegyensúlyozás:** diagonális hasonlósági transzformációval (MATLAB: `balance`) javítható a sajátérték-feladat kondíciója.
- **Hasonlósági transzformáció:**  $P^{-1}AP$  alakkal numerikusan kedvezőbb alak állítható elő.
- **Perturbációanalízis:** az érzékenység előrejelzése segít a megfelelő (pl. szimmetriát kihasználó) algoritmus megválasztásában.

Mindez kulcsfontosságú a dinamikus rendszerek stabilitásvizsgálatában, a PCA/spektrális módszerek konvergenciájában és a hálózatok (Laplace-mátrix) elemzésében.

## 13. LR transzformáció

Az LR transzformáció H. Rutishauser 1959-es, az LR (LU) felbontásra épülő iteratív eljárása egy mátrix **sajátértékeinek** meghatározására. Az  $A_1 = A$  mátrixból indulva mátrixsorozatot állít elő, amelynek főátlója a sajátértékekhez tart.

### Az iteráció

$$A_k = L_k R_k, \quad A_{k+1} = R_k L_k,$$

*Az LR iteráció felbontási lépése*

ahol  $L_k$  alsó háromszögmátrix csupa 1-es főátlóval,  $R_k$  felső háromszögmátrix. Minden lépésben két művelet van:

$$A_k = L_k R_k \quad \Rightarrow \quad A_{k+1} = R_k L_k.$$

Azaz felbontjuk a mátrixot, majd a két tényezőt **fordított sorrendben** összeszorozzuk:

$$A_{k+1} = R_k L_k.$$

*A rekombináció:  $A_{k+1} = R_k \cdot L_k$*

### Miért működik?

Mivel  $R_k = L_k^{-1} A_k$ , ezért  $A_{k+1} = L_k^{-1} A_k L_k$  — tehát minden iterált mátrix **hasonló** az előzőhöz, így a sajátértékek végig megmaradnak. A  $T_k = L_1 \dots L_k$  jelöléssel  $A_{k+1} = T_k^{-1} A_1 T_k$ , és igazolható az  $A_1^k = T_k U_k$  azonosság is.

**Konvergenciátétel.** Ha a trianguláris felbontás minden lépésben létezik, és a sajátértékek abszolút értékben szigorúan elkülönülnek,

$$|\lambda_1| > |\lambda_2| > \dots > |\lambda_n| > 0,$$

akkor az  $A_k$  alsó háromszög része eltűnik, és a főátló a sajátértékekhez konvergál (nagyság szerint csökkenő sorrendben).

### Kidolgozott példa

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}.$$

*A kiindulási mátrix*

Első iteráció — LU felbontás:

$$L_1 = \begin{bmatrix} 1 & 0 \\ 0.5 & 1 \end{bmatrix}, \quad R_1 = \begin{bmatrix} 2 & 1 \\ 0 & 1.5 \end{bmatrix}.$$

*Az  $A = L \cdot R$  felbontás*

Rekombináció:

$$A_1 = R_1 L_1 = \begin{bmatrix} 2 & 1 \\ 0 & 1.5 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ 0.5 & 1 \end{bmatrix} = \begin{bmatrix} 2.5 & 1 \\ 0.5 & 1.25 \end{bmatrix}.$$

Az új iterált,  $A_1 = R \cdot L$

Második iteráció:

$$A_2 = \begin{bmatrix} 2.56 & 1 \\ 0.25 & 1.24 \end{bmatrix}.$$

A következő lépés

Az iterációkat folytatva a főátló a sajátértékekhez konvergál.

## Tulajdonságok és kapcsolat a QR-algoritmussal

- **Konvergencia:** annál gyorsabb, minél jobban elkülönülnek a sajátértékek (nagy spektrális rés).
- **Stabilitás:** nem mindig garantált — közeli sajátértékeknél vagy hiányzó LU-felbontásnál problémás lehet (pivotálás nélkül akár meg is hiúsulhat).
- **Kapcsolat:** az LR transzformáció a **QR-algoritmus előfutára**. A QR ugyanezt az ötletet ortogonális (unitér) hasonlósági transzformációval valósítja meg, ezért numerikusan jóval stabilabb — a gyakorlatban ezért a QR-algoritmust használják. Az LR azonban egyszerűsége miatt kiváló szemléltető eszköz.

## 14. Lineáris egyenletrendszerek iterációs módszerei, a Jacobi iteráció

Kis és közepes (vagy sávszerkezetű) rendszerekre a direkt módszerek (Gauss, LU) ajánlottak; **nagy**, nem feltétlen ritka rendszerekre azonban ezek műveletigénye kezelhetetlenül nagy lehet. Ilyenkor **iterációs** eljárásokat használunk: az  $x$  közelítést lépésről lépésre javítjuk a kívánt pontosságig.

### Az iterációs módszerek általános alakja

$$x^{(k+1)} = Gx^{(k)} + c,$$

Az általános iteráció:  $x = Gx + c$

ahol  $G$  (a továbbiakban  $B$ ) az iterációs mátrix,  $c$  konstans vektor.

### A Jacobi iteráció

Az  $A$  mátrixot a  $D$  átlóra, valamint az alsó ( $L$ ) és felső ( $U$ ) háromszög részre bontjuk:

$$A = D + L + U,$$

Az  $A = D - L - U$  felbontás

Innen az  $Ax = b$  egyenletet átrendezve adódik a Jacobi-féle iterációs alak:

$$Dx = b - (L + U)x.$$

Az átrendezett egyenlet

$$x^{(k+1)} = D^{-1}(b - (L + U)x^{(k)}).$$

Az iterációs képlet

Tömör jelöléssel  $x^{(k+1)} = D^{-1}b - D^{-1}(A-D)x^{(k)} = Bx^{(k)} + c$ , komponensenként pedig:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left( b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right).$$

Az  $i$ -edik ismeretlen frissítése

A lényeg: minden komponens frissítése csak az **előző** iteráció értékeit használja, ezért a lépések egymástól függetlenek (jól párhuzamosíthatók). Az iterációt addig folytatjuk, amíg két közelítés eltérése egy  $\epsilon$  tűrés alá nem csökken:

$$\|x^{(k+1)} - x^{(k)}\| < \epsilon.$$

Megállási feltétel

### Kidolgozott példa

$$\begin{aligned} 4x_1 + x_2 &= 15, \\ x_1 + 3x_2 &= 10. \end{aligned}$$

A megoldandó egyenletrendszer

$$A = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix}, \quad b = \begin{bmatrix} 15 \\ 10 \end{bmatrix}.$$

A mátrixok

$$D = \begin{bmatrix} 4 & 0 \\ 0 & 3 \end{bmatrix}, \quad L + U = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

A felbontás

$$x_1^{(k+1)} = \frac{1}{4} (15 - x_2^{(k)}), \quad x_2^{(k+1)} = \frac{1}{3} (10 - x_1^{(k)}).$$

Az iterációs képlet

$$x^{(0)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Induló közelítés

$$x_1^{(1)} = \frac{1}{4}(15 - 0) = 3.75, \quad x_2^{(1)} = \frac{1}{3}(10 - 0) = 3.33.$$

Első iteráció

$$x_1^{(2)} = \frac{1}{4}(15 - 3.33) = 2.9175, \quad x_2^{(2)} = \frac{1}{3}(10 - 3.75) = 2.0833.$$

Második iteráció

Figyelem: a Jacobi iteráció érzékeny az egyenletek sorrendjére. Ugyanaz a rendszer más sorrendben felírva (ha a diagonális dominancia elromlik) **divergálhat** — a slide példájában az utolsó sort előrevéve a vektorsorozat néhány lépés alatt elszáll.

## Konvergencia

A Jacobi iteráció akkor konvergál minden indulóvektorra, ha az iterációs mátrix spektrálsugara 1-nél kisebb,  $\rho(B) < 1$ . Elegendő (de nem szükséges) feltétel a **szigorú diagonális dominancia**:

$$\sum_{j \neq i} |a_{ij}| < |a_{ii}|, \quad i = 1, \dots, n,$$

ami épp azt jelenti, hogy  $\|B\|_{\infty} = \|D^{-1}(A-D)\|_{\infty} < 1$ .

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|, \quad \forall i.$$

*Diagonális dominancia*

$$(G = D^{-1}(L + U))$$

*Az iterációs mátrix*

$$\rho(G) < 1.$$

*$\rho(B) < 1$  feltétel*

## Előnyök és hátrányok

- **Előny:** egyszerű, jól párhuzamosítható (a komponensek függetlenek), ritka mátrixokra memóriahatékony.
- **Hátrány:** lassú konvergencia, különösen, ha  $A$  nem diagonálisan domináns; gyakran sok iteráció kell. A Gauss–Seidel iteráció rendszerint gyorsabb.

## 15. Lineáris egyenletrendszerek iterációs módszerei konvergenciája

A konvergencia azt jelenti, hogy az  $x^{(k)}$  közelítőssorozat a pontos  $x^*$  megoldáshoz tart. Ez az  $x^{(k+1)} = Bx^{(k)} + c$  iteráció  $B$  mátrixának spektrális tulajdonságain múlik.

$$\lim_{k \rightarrow \infty} x^{(k)} = x.$$

*A közelítések a megoldáshoz tartanak*

$$G = I - D^{-1}A,$$

*Az iterációs mátrix*

### A konvergencia alaptétele

Jelölje  $e_k = x^{(k)} - x^*$  a hibavektort. Mivel  $x^*$  is kielégíti az iterációt,  $e_{k+1} = Be_k$ , azaz

$$e_k = B^k e_0.$$

**Tétel.** Az iteráció pontosan akkor globálisan konvergens (bármely indulóvektorra ugyanahhoz a megoldáshoz tart), ha a spektrálsugár 1-nél kisebb:

$$\rho(B) < 1.$$

A bizonyítás kulcsa, hogy  $\rho(B) < 1$  esetén létezik olyan mátrixnorma, amelyre  $\|B\| < 1$ , és ekkor  $\|e_k\| \leq \|B\|^k \|e_0\| \rightarrow 0$ . (Speciális eset: ha  $B$  szigorúan trianguláris, akkor nilpotens, és az iteráció **véges** sok lépésben pontos.)

### Elegendő, könnyen ellenőrizhető feltételek

- **Diagonális dominancia:** ha minden sorra teljesül, az iteráció garantáltan konvergál.

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|.$$

*Diagonális dominancia*

$$A = \begin{bmatrix} 4 & 1 & 0 \\ 2 & 6 & 1 \\ 0 & 2 & 5 \end{bmatrix}.$$

*Diagonálisan domináns mátrix*

- **Spektrálsugár:** a tényleges (szükséges és elegendő) feltétel  $\rho(B) < 1$ .

$$\rho(G) < 1.$$

$\rho(G) < 1$

$$\rho(G) = \max(|\lambda_1|, |\lambda_2|, \dots, |\lambda_n|).$$

$\lambda$

- **Szimmetria és pozitív definités:** szimmetrikus, pozitív definit  $A$  esetén a Gauss–Seidel (és a konjugált gradiens) konvergál.

## A konvergencia sebessége

Az iterációs módszerek jellemzően **lineárisan** konvergálnak: a hiba csökkenése arányos az előzővel, az arány nagyjából  $\rho(B)$ .

$$\|x^{(k+1)} - x\| \leq \rho(G) \|x^{(k)} - x\|.$$

*Lineáris konvergencia*

Minél kisebb  $\rho(B)$ , annál gyorsabb a konvergencia; ha  $\rho(B)$  közel 1, az iteráció nagyon lassú.

*Példa.*

$$4x_1 + x_2 = 15, \quad x_1 + 3x_2 = 10.$$

*A lineáris rendszer*

$$G = \begin{bmatrix} 0 & -0.25 \\ -0.333 & 0 \end{bmatrix}.$$

*A Jacobi iterációs mátrix*

$$\lambda_1 = 0, \lambda_2 = -0.0833.$$

*A  $G$  sajátértékei*

Mivel  $\rho(G) = 0,0833 < 1$ , a Jacobi iteráció konvergál, méghozzá gyorsan.

## Jacobi és Gauss–Seidel összehasonlítása

| Módszer      | Konvergencia feltételek                            | Sebesség |
|--------------|----------------------------------------------------|----------|
| Jacobi       | Diagonális dominancia vagy $\rho(G) < 1$           | Lassabb  |
| Gauss–Seidel | Gyorsabban konvergál diagonális dominancia mellett | Gyorsabb |

*Jacobi és Gauss–Seidel összehasonlítása*

A Gauss–Seidel rendszert **gyorsabban** konvergál, mert az újonnan kiszámított komponenseket azonnal felhasználja; a Jacobi viszont **párhuzamosítható**, ami nagy rendszereknél előny. Mindkettő a végelem-analízisben és nagy, ritka rendszerek megoldásában tipikus.



## 16. A Gauss–Seidel iteráció, mátrixok reguláris szétvágásai

### A Gauss–Seidel iteráció

A Gauss–Seidel iteráció a Jacobihoz hasonlóan az  $A = D - L - U$  felbontásra épül, de egy lényeges különbséggel: az aktuális lépésben **már kiszámított** komponenseket azonnal felhasználja.

$$A = D + L + U.$$

Az  $A = D - L - U$  felbontás

$$x^{(k+1)} = -(D + L)^{-1}Ux^{(k)} + (D + L)^{-1}b.$$

A Gauss–Seidel iterációs képlet

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left( b_i - \sum_{j < i} a_{ij}x_j^{(k+1)} - \sum_{j > i} a_{ij}x_j^{(k)} \right).$$

Az  $i$ -edik ismeretlen frissítése

- A **Jacobi** csak az előző iteráció  $x_j^{(k)}$  értékeit használja.
- A **Gauss–Seidel** az ugyanabban a lépésben már frissített  $x_j^{(k+1)}$  értékeket is.

Ez rendszerint gyorsabb konvergenciát ad, cserébe a számítás soros (nehezen párhuzamosítható). A megállási feltétel a szokásos:

$$\|x^{(k+1)} - x^{(k)}\| < \epsilon,$$

Megállási feltétel

Példa.

$$\begin{aligned} 4x_1 + x_2 &= 15, \\ x_1 + 3x_2 &= 10. \end{aligned}$$

A megoldandó rendszer

$$x_1^{(k+1)} = \frac{1}{4}(15 - x_2^{(k+1)}), \quad x_2^{(k+1)} = \frac{1}{3}(10 - x_1^{(k+1)}).$$

A frissítési képletek

$$x^{(0)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Induló közelítés

$$x_1^{(1)} = \frac{1}{4}(15 - 0) = 3.75, \quad x_2^{(1)} = \frac{1}{3}(10 - 3.75) = 2.0833.$$

*Első iteráció*

$$x_1^{(2)} = \frac{1}{4}(15 - 2.0833) = 3.2292, \quad x_2^{(2)} = \frac{1}{3}(10 - 3.2292) = 2.2569.$$

*Második iteráció*

## Mátrixok reguláris szétvágása

Az iterációs alakhoz az  $A$ -t  $A = R - S$  alakban bontjuk, ahol  $R$  reguláris. Ez a **reguláris szétvágás**, amelyből:

$$A = M - N,$$

$A = M - N$  reguláris szétvágás

$$x^{(k+1)} = M^{-1}Nx^{(k)} + M^{-1}b.$$

*Az iterációs forma*

azaz  $x^{(k+1)} = Bx^{(k)} + c$ , ahol  $B = R^{-1}S = I - R^{-1}A$  és  $c = R^{-1}b$ . A két klasszikus módszer egy-egy szétvágás (a  $\hat{L} = D^{-1}L$ ,  $\hat{U} = D^{-1}U$  jelöléssel):

$$M = D, \quad N = -(L + U).$$

*Jacobi:  $R = D$ ,  $S = L + U$*

$$M = D + L, \quad N = -U.$$

*Gauss–Seidel:  $R = D - L$ ,  $S = U$*

- **Jacobi:**  $B = D^{-1}(L+U) = \hat{L} + \hat{U}$ .
- **Gauss–Seidel:**  $B = (D-L)^{-1}U = (I-\hat{L})^{-1}\hat{U}$ .

## Konvergencia

Erősen reguláris  $A$  esetén az iteráció pontosan akkor konvergens, ha  $\rho(B) < 1$  (illetve ha van olyan norma, amelyre  $\|B\| < 1$ ).

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|.$$

*Diagonális dominancia*

$$\rho(M^{-1}N) < 1.$$

$\rho(M^{-1}N) < 1$

**Tétel.** Ha létezik olyan mátrixnorma, amelyre  $\|\hat{L}\| + \|\hat{U}\| < 1$ , akkor **mind** a Jacobi, **mind** a Gauss–Seidel iteráció konvergens. (Jacobiánál  $\|B\| = \|\hat{L} + \hat{U}\| \leq \|\hat{L}\| + \|\hat{U}\| < 1$ ; Gauss–Seidelnél  $\|e_{k+1}\| \leq \|\hat{U}\|/(1 - \|\hat{L}\|) \cdot \|e_k\|$ .) Mivel minden diagonálisan domináns mátrix erősen reguláris, a diagonális dominancia mindkét

módszer konvergenciáját biztosítja.

## Előnyök és hátrányok

- **Előny:** gyorsabb konvergencia, mint a Jacobié; az új értékek azonnali felhasználása felgyorsítja az iterációt.
- **Hátrány:** nehezen párhuzamosítható (soros függőség); rosszul kondicionált vagy nem domináns mátrixoknál lassú.

## 17. Konjugált gradiens módszer

A konjugált gradiens (Conjugate Gradient, CG) módszer **szimmetrikus, pozitív definit** mátrixú lineáris rendszerek iteratív megoldására szolgál. Pontos aritmetikával véges sok (legfeljebb  $n$ ) lépésben megtalálná a megoldást, de a kerekítési hibák miatt iterációként kezeljük. Nagy, ritka rendszerekre kiválóan alkalmas.

$$Ax = b,$$

*A megoldandó rendszer (A szimmetrikus, pozitív definit)*

### Alapötlet: optimalizálás

A megoldást egy kvadratikus függvény minimumaként fogjuk fel:

$$q(x) = \frac{1}{2}x^T Ax - x^T b.$$

$$f(x) = \frac{1}{2}x^T Ax - b^T x.$$

*A kvadratikus célfüggvény*

Mivel  $A$  pozitív definit,  $q$  paraboloid, amelynek egyetlen minimuma épp az  $Ax^* = b$  megoldás. Két kulcsészrevétel:

- A negatív gradiens a **reziduum**:  $-\nabla q(x) = b - Ax = r$ .
- Adott  $s_k$  keresési irány mentén az optimális lépéshossz **zárt alakban** adódik (ott, ahol az új reziduum merőleges  $s_k$ -ra).

### Konjugált irányok

A megoldást nem tetszőleges, hanem egymásra **A-konjugált** irányok mentén keressük ( $p_i^T A p_j = 0$ , ha  $i \neq j$ ):

$$p_i^T A p_j = 0, \quad \forall i \neq j.$$

*Az irányok A-konjugáltsága*

### Az algoritmus

Induljunk az  $x_0$  pontból, legyen  $s_0 = r_0 = b - Ax_0$ , majd ismételjük:

$$\alpha_k = \frac{r_k^T r_k}{s_k^T A s_k}, \quad x_{k+1} = x_k + \alpha_k s_k, \quad r_{k+1} = r_k - \alpha_k A s_k$$

$$\beta_{k+1} = \frac{r_{k+1}^T r_{k+1}}{r_k^T r_k}, \quad s_{k+1} = r_{k+1} + \beta_{k+1} s_k$$

$$r^{(0)} = b - Ax^{(0)}.$$

*A kezdeti reziduum*

$$p^{(0)} = r^{(0)}.$$

A kezdeti irány

$$\alpha_k = \frac{(r^{(k)})^T r^{(k)}}{(p^{(k)})^T A p^{(k)}}.$$

A lépéshossz

$$x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)}.$$

Az új közelítés

$$r^{(k+1)} = r^{(k)} - \alpha_k A p^{(k)}.$$

Az új reziduum

$$\beta_k = \frac{(r^{(k+1)})^T r^{(k+1)}}{(r^{(k)})^T r^{(k)}}.$$

A  $\beta$  együttható

$$p^{(k+1)} = r^{(k+1)} + \beta_k p^{(k)}.$$

Az új keresési irány

Az iterációt addig folytatjuk, amíg a reziduum normája elég kicsi:

$$\|r^{(k+1)}\| < \epsilon.$$

Megállási feltétel

A módszer neve onnan ered, hogy a sima gradiensmódszer „völgyszerű” függvényeknél feleslegesen sokat cikázik; a CG a lépésenkénti konjugált irányváltással ezt kiküszöböli, és a hibát minden lépésben egy újabb  $A$ -ortogonális irányban tünteti el.

## Kidolgozott példa

$$\begin{aligned} 4x_1 + x_2 &= 15, \\ x_1 + 3x_2 &= 10. \end{aligned}$$

A rendszer

$$A = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix}, \quad b = \begin{bmatrix} 15 \\ 10 \end{bmatrix}.$$

A mátrixok

$$\mathbf{x}^{(0)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad \mathbf{r}^{(0)} = \mathbf{b} - \mathbf{A}\mathbf{x}^{(0)} = \begin{bmatrix} 15 \\ 10 \end{bmatrix}.$$

*Induló értékek*

$$\alpha_0 = \frac{(\mathbf{r}^{(0)})^T \mathbf{r}^{(0)}}{(\mathbf{p}^{(0)})^T \mathbf{A} \mathbf{p}^{(0)}} = \frac{15^2 + 10^2}{15 \cdot (4 \cdot 15 + 10) + 10 \cdot (15 + 30)} = 0.27.$$

*A lépéshossz*

$$\mathbf{x}^{(1)} = \mathbf{x}^{(0)} + \alpha_0 \mathbf{p}^{(0)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} + 0.27 \cdot \begin{bmatrix} 15 \\ 10 \end{bmatrix} = \begin{bmatrix} 4.05 \\ 2.7 \end{bmatrix}.$$

*Az új közelítés*

Az iterációkat folytatva a megoldás (legfeljebb  $n$  lépésben) konvergál:

$$\mathbf{x} = \begin{bmatrix} 3 \\ 2 \end{bmatrix}.$$

*A megoldás*

## Előnyök és hátrányok

- **Előny:** nagy, ritka SPD rendszerekre nagyon hatékony; gyorsabb a Jacobi/Gauss–Seidel módszereknél; elvileg  $\leq n$  lépésben pontos.
- **Hátrány:** csak szimmetrikus, pozitív definit mátrixra; a kerekítési hibák ronthatják a pontosságot (a gyakorlatban előkondicionálással, pcg-vel javítható).

## 18. Lineáris egyenletek iterációs módszerei a MATLAB-ban

A MATLAB-ban az iterációs módszerek néhány soros, vektorizált kóddal megvalósíthatók, a konjugált gradienshez pedig kész, beépített függvény is van. A tömör mátrix–vektor írásmód (pl. egy sor főátlón kívüli elemeinek indexelése) miatt a kód rövid és gyors marad.

### Jacobi iteráció

Az  $AX = B$  rendszert oldja meg a  $P$  kezdővektorból, szigorúan diagonálisan domináns  $A$  esetén konvergálva. Minden komponenst kizárólag az **előző** iteráció értékeiből frissít:

matlab

Kód másolása

```
% Mátrix és vektor definiálása
A = [4, 1; 1, 3];
b = [15; 10];
x0 = [0; 0]; % Kezdeti közelítés
tol = 1e-6; % Konvergencia feltétel
maxIter = 100; % Maximális iterációk száma

D = diag(diag(A)); % Diagonális mátrix
L_plus_U = A - D; % Felső és alsó háromszög elemek
x = x0;

for k = 1:maxIter
    x_new = D \ (b - L_plus_U * x); % Jacobi iterációs képlet
    if norm(x_new - x, inf) < tol % Konvergencia ellenőrzése
        break;
    end
    x = x_new;
end

disp('Megoldás:');
disp(x);
```

*jacobi(A,B,P,delta,max1) — kézi implementáció*

A delta a kívánt tűrés, a max1 az iterációk maximális száma. Érdekes megfigyelné a gépi pontosság (eps) használatát a relatív hiba számításánál, és hogy az  $A(j,[1:j-1,j+1:N])$  a  $j$ -edik sor főátlón kívüli elemeit adja.

### Gauss–Seidel iteráció

Szerkezete csaknem azonos a Jacobiéval — csak a belső ciklus magjában tér el: a már kiszámított  $X(1:j-1)$  új értékeket azonnal felhasználja, ezért rendszerint gyorsabban konvergál:

```

% Mátrix és vektor definiálása
A = [4, 1; 1, 3];
b = [15; 10];
x0 = [0; 0]; % Kezdeti közelítés
tol = 1e-6; % Konvergencia feltétel
maxIter = 100; % Maximális iterációk száma

L_plus_D = tril(A); % Alsó háromszög és diagonális mátrix
U = A - L_plus_D; % Felső háromszög elemek
x = x0;

for k = 1:maxIter
    x_new = L_plus_D \ (b - U * x); % Gauss-Seidel iterációs képlet
    if norm(x_new - x, inf) < tol % Konvergencia ellenőrzése
        break;
    end
    x = x_new;
end

disp('Megoldás:');
disp(x);

```

*gseidel(A,B,P,delta,max1) — kézi implementáció*

## Konjugált gradiens módszer

A MATLAB-nak van beépített, előkondicionált konjugált gradiens függvénye, a **pcg** (Preconditioned Conjugate Gradient), amely szimmetrikus, pozitív definit  $A$ -ra való:

```

% Mátrix és vektor definiálása
A = [4, 1; 1, 3];
b = [15; 10];
x0 = [0; 0]; % Kezdeti közelítés
tol = 1e-6; % Konvergencia feltétel
maxIter = 100; % Maximális iterációk száma

% Konjugált gradiens módszer
[x, flag, relres, iter] = pcg(A, b, tol, maxIter, [], [], x0);

disp('Megoldás:');
disp(x);
disp(['Iterációk száma: ', num2str(iter)]);

```



A módszer kézzel is néhány sorban megírható (a lépéshossz, a reziduum és a konjugált irány frissítésével):

matlab

Kód másolása

```
% Mátrix és vektor definiálása
A = [4, 1; 1, 3];
b = [15; 10];
x = [0; 0]; % Kezdeti közelítés
r = b - A * x; % Kezdeti maradék
p = r;
tol = 1e-6; % Konvergencia feltétel

for k = 1:100
    alpha = (r' * r) / (p' * A * p);
    x_new = x + alpha * p;
    r_new = r - alpha * A * p;
    if norm(r_new) < tol
        break;
    end
    beta = (r_new' * r_new) / (r' * r);
    p = r_new + beta * p;
    x = x_new;
    r = r_new;
end

disp('Megoldás:');
disp(x);
```

Manuális CG-implementáció

## A módszerek összehasonlítása

| Módszer            | Sebesség              | Alkalmazás                                        |
|--------------------|-----------------------|---------------------------------------------------|
| Jacobi             | Lassúbb               | Ritka mátrixokhoz, egyszerű implementáció         |
| Gauss-Seidel       | Gyorsabb, mint Jacobi | Nagyobb mátrixok esetén jobban teljesít           |
| Konjugált gradiens | Leggyorsabb           | Szimmetrikus, pozitív definit mátrixokhoz ideális |

A módszerek összehasonlítása

A módszer megválasztása a feladattól függ: a Jacobi jól párhuzamosítható, a Gauss–Seidel gyorsabban konvergál, a (pre)konjugált gradiens pedig nagy, ritka, szimmetrikus pozitív definit rendszerekre a leghatékonyabb — tipikusan mérnöki szimulációkban (áramlástan, mechanika, végelelem-analízis) és kvadratikus optimalizálásban.

## 19. Polinomok zérushelyei, Horner-elrendezés, Ruffini-sorozat, iterált Horner-elrendezés

### Polinomok zérushelyei

A  $p(x) = a_0x^n + a_1x^{n-1} + \dots + a_n$  polinomokkal bonyolultabb függvényeket közelítünk; könnyű őket deriválni, integrálni és gyökeiket közelíteni. Az  $x_0$  a  $p$  **zérushelye** (gyöke), ha  $p(x_0) = 0$ :

$$P(x_0) = 0.$$

$$p(x_0) = 0$$

$$P(x) = a_nx^n + a_{n-1}x^{n-1} + \dots + a_1x + a_0,$$

*n-edfokú polinom általános alakja*

Ez pontosan akkor teljesül, ha  $p(x) = (x - x_0)q(x)$ . Fontos állítások: egy  $n$ -edfokú polinomnak multiplicitással számolva **pontosan  $n$**  komplex gyöke van (gyöktényezős alak:  $p = a_0 \prod (x - x_i)$ ); valós együtthatós polinom komplex gyökei konjugált párokban állnak; és — az Abel–Ruffini-tétel értelmében — **nincs** általános, gyökvonással felírt megoldóképlet ötöd- és magasabb fokra (ezért kellenek numerikus módszerek: Newton, Bairstow).

### Horner-elrendezés

A polinomot kiértékeléshez érdemes átrendezni, hogy minimalizáljuk a műveletszámot:

$$p(x) = (\dots((a_0x + a_1)x + a_2)x + \dots + a_{n-1})x + a_n.$$

$$P(x) = a_nx^n + a_{n-1}x^{n-1} + \dots + a_1x + a_0,$$

*Általános polinom*

$$P(x) = (\dots((a_nx + a_{n-1})x + a_{n-2})x + \dots + a_1)x + a_0.$$

*Horner-alak*

Ez a séma egy  $n$ -edfokú polinom kiértékeléséhez mindössze  **$n$  szorzást és  $n$  összeadást** igényel — a kifejített alaknál lényegesen kevesebbet.

$$P(x) = 2x^3 - 6x^2 + 2x - 1.$$

*Példa polinom*

$$P(x) = ((2x - 6)x + 2)x - 1.$$

*Horner-alakja*

$$P(3) = ((2 \cdot 3 - 6) \cdot 3 + 2) \cdot 3 - 1 = (6 - 6) \cdot 3 + 2 \cdot 3 - 1 = 5.$$

*Kiértékelés  $x = 3$ -ra*

## Ruffini-sorozat (szintetikus osztás)

A Horner-elrendezéssel egyenértékű, gépen könnyen megvalósítható az  $x^*$  helyhez tartozó Ruffini-sorozat:

$$b_0 = a_0, \quad b_k = b_{k-1}\hat{x} + a_k \quad (k = 1, \dots, n).$$

**Segédttétel.** A  $q(x) = b_0x^{n-1} + \dots + b_{n-1}$  polinommal:

$$p(x) = (x - \hat{x})q(x) + b_n, \quad p(\hat{x}) = b_n, \quad p'(\hat{x}) = q(\hat{x}).$$

Vagyis a sorozat egyszerre adja a  $p(x)$  helyettesítési értéket és az  $x - \hat{x}$ -vel való osztás hányadosát; mindez  $O(n)$  művelet, és így  $O(n)$  lépésben eldönthető, hogy  $x^*$  gyök-e.

$$P(x) = 2x^3 - 6x^2 + 2x - 1, \quad \text{osztó: } x - 3.$$

*Osztás  $x - c$ -vel*

|   |   |    |   |    |
|---|---|----|---|----|
| 3 | 2 | -6 | 2 | -1 |
|   |   | 6  | 0 | 6  |
|   | 2 | 0  | 2 | 5  |

*A Ruffini-táblázat*

$$Q(x) = 2x^2 + 0x + 2, \quad \text{maradék: } 5.$$

*Az eredmény*

## Iterált Horner-elrendezés

Ha nemcsak a helyettesítési értékre, hanem a deriváltakra is szükség van, az iterált Horner-elrendezést használjuk: a Ruffini-sorozatot ismételten alkalmazva táblázatot építünk, amelynek „lecsippentett” elemei a polinom  $x - x^*$  hatványai szerinti átrendezett alakját adják. A Taylor-polinom egyértelműsége miatt ezek épp a deriváltértékek:

$$\frac{p^{(k)}(\hat{x})}{k!} = a_{n-k+1}, \quad k = 0, 1, \dots, n.$$

$$P(x) = 2x^3 - 6x^2 + 2x - 1.$$

*Példa polinom*

$$[2, -6, 2, -1].$$

*A táblázat első sora*

$$P'(x_0) = ((6)x + 0)x + 2.$$

*A második sor*

$$P(3) = 5, \quad P'(3) = 20.$$

*Az eredmény:  $p(x^*)$  és  $p'(x^*)$*

Így a  $p(x)$ ,  $p'(x)$ , ...,  $p^{(n)}(x)$  értékek összesen  $O(n^2)$  művelettel számíthatók, és egy gyök multiplicitása is meghatározható (a derivált-sorozat végén álló nullák megszámlálásával).

## Alkalmazások

- **Kiértékelés:** a Horner-elrendezés gyorsítja a polinomok kiértékelését (lényeges sűrűn hívott szubrutinban).
- **Osztás:** a Ruffini-sorozat  $x - c$  alakú osztóra szintetikus osztást ad.
- **Gyökkeresés:** az iterált Horner a  $p$  és  $p'$  egyidejű előállításával közvetlenül támogatja a Newton-módszert.

## 20. Polinomok kezelése a MATLAB-ban

A MATLAB a polinomot az együtthatóinak **sorvektorával** ábrázolja, a legmagasabb fokú tagtól kezdve (a nulla együtthatókat is meg kell adni). Például a  $2x^3 + 2x + 3$  polinom a `[2 0 2 3]` vektor.

$$P(x) = 2x^3 - 5x^2 + 3x - 1$$

*Egy polinom*

matlab

 Kód másolása

```
P = [2 -5 3 -1];
```

*Reprezentáció együtthatóvektorral*

### Kiértékelés és gyökök

A **polyval** a Horner-elrendezés alapján értékeli ki egy adott pontban; a **roots** a gyököket adja (egy alkalmasan összeállított **kísérőmátrix** — companion matrix — sajátértékeiként!), a **poly** pedig a gyökökből (vagy egy mátrix karakterisztikus polinomjaként) állítja elő az együtthatókat.

matlab

 Kód másolása

```
x = 2; % Az érték, ahol a polinomot ki kell értékelni  
y = polyval(P, x);
```

*polyval — kiértékelés*

matlab

 Kód másolása

```
roots_P = roots(P);
```

*roots — gyökök*

Érdekesség: a `roots.m` valójában a gyökkeresést sajátérték-feladatra vezeti vissza (eig a kísérőmátrixon). A `poly` fordítva működik, de mindig 1 főegyütthatójú polinomot ad, így eltérhet a kiindulástól.

### Deriválás és integrálás

A **polyder** a deriváltat, a **polyint** az integrált adja — ezek nem szimbolikus műveletek, csak a polinom-együtthatókra alkalmazott deriválási/integrálási szabályok.

matlab

 Kód másolása

```
dP = polyder(P);
```

matlab

 Kód másolása

```
iP = polyint(P);
```

polyint — integrálás

## Szorzás és osztás

A polinomszorzás a **conv** (konvolúció), a (maradékos) osztás a **deconv** utasítással történik. Összeadásra/kivonásra nincs külön parancs — azt a sorvektorok megfelelő hossza igazítása után egyszerű vektorművelettel végezzük.

matlab

 Kód másolása

```
Q = [1 -2 1]; % Egy másik polinom
R = conv(P, Q);
```

conv — szorzás

matlab

 Kód másolása

```
[Q, R] = deconv(P, Q); % Q a hányados, R a maradék
```

deconv — osztás

## Ábrázolás

A polinom görbéje a polyval és a plot (gyakran linspace alappontokkal) segítségével rajzolható ki.

matlab

 Kód másolása

```
x = linspace(-10, 10, 100); % Értékkészlet
y = polyval(P, x);
plot(x, y);
xlabel('x');
ylabel('P(x)');
title('Polinom grafikus ábrázolása');
grid on;
```

Grafikus ábrázolás

```
% Polinom definíció
P = [2 -5 3 -1];

% Polinom értéke x = 1.5-nél
val = polyval(P, 1.5);

% Polinom deriváltja és gyökei
dP = polyder(P);
roots_P = roots(P);

% Polinom ábrázolása
x = linspace(-2, 3, 100);
y = polyval(P, x);
plot(x, y);
xlabel('x');
ylabel('P(x)');
title('Polinom ábrázolása');
grid on;
```

*Konkrét MATLAB-példa*

## 21. Függvényközelítések, Lagrange-interpoláció

Megjegyzés: az eredeti dokumentumban ez a tétel tévesen a 22. tétel (a Lagrange-interpoláció hibája) szövegét ismételte. Az alábbi, helyes tartalmat az előadás főlíái (81–85.) alapján pótoltuk.

### Függvényközelítés és interpoláció

Mivel a polinomokkal gyorsan és hatékonyan számolhatunk (és könnyen deriválhatók, integrálhatók), bonyolultabb függvények közelítésére is őket használjuk. Adott  $(x_i, y_i)$ ,  $i = 0, \dots, n$  pontsorozathoz azt a függvényt keresni, amely **minden ponton átmegy, interpoláció**; ha a keresett függvény polinom, polinominterpolációról beszélünk. Fogalmak:

- **Interpoláció vs. extrapoláció:** az  $x$  becslési pont az alappontok  $[\min, \max]$  tartományán belül (interpoláció), illetve azon kívül (extrapoláció) van.
- **Spline:** több alacsony fokszámú polinomból összerakott, a csatlakozási pontokon előírt deriváltakkal illeszkedő függvény.
- **Görbeillesztés:** ha a polinom fokszáma kisebb, mint  $m - 1$ , és nem feltétlen megy át minden ponton.

A polinominterpolációnál a fokszám  $n = m - 1$  (eggyel kevesebb, mint a pontok száma).

### Létezés és egyértelműség (Vandermonde)

Az interpolációs feltételek lineáris egyenletrendszernek adnak az együtthatókra, amelynek mátrixa a **Vandermonde-mátrix**:

$$\sum_{k=0}^n a_k x_i^k = y_i, \quad i = 0, 1, \dots, n.$$

Determinánsa a páronkénti különbségek szorzata:

$$\det V = \prod_{i>j} (x_i - x_j).$$

Ezért ha az alappontok **páronként különbözők**, a determináns nem nulla, tehát **pontosan egy** legfeljebb  $n$ -edfokú interpoláló polinom létezik.

### A Lagrange-interpolációs polinom

A Lagrange-féle alak az alappolinomokra építve közvetlenül felírja a megoldást:

$$p_n(x) = \sum_{i=0}^n f(x_i) L_i(x), \quad L_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}.$$

Az  $L_i$  alappolinomok kulcstulajdonsága, hogy  $L_i(x_j) = 1$ , ha  $i = j$ , és 0 egyébként. Ebből azonnal adódik, hogy  $p_n(x_j) = f(x_j)$ , tehát a polinom valóban átmegy az összes adatponton. Az egyértelműség abból következik, hogy két ilyen polinom különbsége legfeljebb  $n$ -edfokú, de  $n + 1$  gyöke van, tehát azonosan nulla.

MATLAB: az együtthatók a `lagran(X,Y)` függvénnyel ( $C = Y \cdot L$ , ahol az  $L$  sorai az alappolinomok együtthatói), illetve a `polyfit(X,Y,n)` paranccsal kaphatók meg.



## Kidolgozott példa

Illesszünk polinomot a  $(0, 0)$ ,  $(1, -1)$ ,  $(2, 0)$ ,  $(3, 1)$ ,  $(4, 0)$  pontokra. A nulla függvényértékű tagok kiesnek, és egyszerűsítés után:

$$p(x) = -\frac{1}{3} x (x - 2)(x - 4).$$

Az 5 pontban vett (extrapolált) érték:  $p(5) = -15/3 = -5$ . (Ugyanezt kapjuk a Vandermonde-rendszer Gauss-eliminációval való megoldásából is:  $a_0 = 0$ ,  $a_1 = -8/3$ ,  $a_2 = 2$ ,  $a_3 = -1/3$ ,  $a_4 = 0$ .)

## 22. A Lagrange-interpoláció hibája, interpoláció a MATLAB-ban

### A Lagrange-interpoláció hibája

Az interpoláló polinom az alappontokban pontosan illeszkedik, de közöttük eltérhet az eredeti függvénytől — ezt nevezzük **interpolációs hibának**. A hibavizsgálathoz feltesszük, hogy  $f$  az alappontokat és az  $x$  pontot tartalmazó legszűkebb  $[a, b]$  intervallumon  $(n+1)$ -szer differenciálható. Ekkor az  $\omega_n(x) = (x-x_0)(x-x_1)\dots(x-x_n)$  jelöléssel a maradéktag:

$$R(x) = f(x) - P(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i),$$

*A Lagrange-interpoláció maradéktagja*

$$f(\hat{x}) - p_n(\hat{x}) = \frac{\omega_n(\hat{x})}{(n+1)!} f^{(n+1)}(\xi), \quad \xi \in [a, b].$$

ahol  $f^{(n+1)}(\xi)$  az eredeti függvény  $(n+1)$ -edik deriváltja egy  $\xi \in [x_0, x_n]$  közbülső pontban. A képlet a Rolle-tétel ismételt alkalmazásával vezethető le (egy ügyesen választott segédfüggvénynek legalább  $n+2$  zérushelye van). Ha  $f^{(n+1)}$  korlátos, felső korlátja  $\mu_{n+1}$ , akkor:

$$|f(\hat{x}) - p_n(\hat{x})| \leq \frac{|\omega_n(\hat{x})|}{(n+1)!} \mu_{n+1} \leq \frac{\max_{x \in [a, b]} |\omega_n(x)|}{(n+1)!} \mu_{n+1}.$$

A hiba tehát három dologtól függ: az  $(n+1)$ -edik derivált nagyságától, az  $x$  alappontoktól mért távolságtól, valamint az alappontok számától és elhelyezkedésétől.

**Runge-jelenség:** egyenletes (ekvidisztáns) alappontoknál magas fokszámon a polinom a tartomány **szélein** erősen oszcillálhat. A hiba csökkenthető **Csebisev-pontok** használatával, amelyek a  $\max |\omega_n(x)|$  értéket minimalizálják, és így letompítják a szélső oszcillációkat.

### Interpoláció a MATLAB-ban

Egy dimenzióban a fő eszköz az **interp1**:

- $y0 = \text{interp1}(x, y, x0, 'típus')$  — az  $(x, y)$  pontokra illeszt és kiértékel  $x0$ -ban (az  $x$ -nek szigorúan növekvőnek kell lennie).
- A típus lehet 'linear' (alapértelmezett), 'nearest', 'cubic' vagy 'spline'.

A 'nearest' durva, lépcsős közelítést ad, a 'spline' viszont sima, kiváló leírást — a lineáris is csak az erősen görbült részekben tér el. (Magasabb dimenzióban: **interp2**, illetve szórt adatra **griddata**, gyakran **meshgrid-del**.) Az interpoláló polinom közvetlenül is előállítható a **polyfit** / **polyval** párossal.

```
matlab Kód másolása

% Adatpontok
x = [0 1 2];
y = exp(-x);

% Polinom illesztése
P = polyfit(x, y, length(x)-1);

% Polinom kiértékelése és hiba számítása
xx = linspace(0, 2, 100); % Sűrűbb pontok az ábrázoláshoz
yy = polyval(P, xx); % Interpolációs polinom kiértékelése
f_true = exp(-xx); % Eredeti függvény

% Hiba kiszámítása
error = abs(f_true - yy);

% Ábrázolás
figure;
subplot(2, 1, 1);
plot(xx, f_true, 'b', xx, yy, 'r--', x, y, 'ko');
legend('Eredeti függvény', 'Interpoláció', 'Adatpontok');
title('Interpoláció és eredeti függvény');
xlabel('x');
ylabel('y');
grid on;

subplot(2, 1, 2);
plot(xx, error, 'm');
title('Interpolációs hiba');
xlabel('x');
ylabel('Hiba');
grid on;
```

MATLAB-példa: interpoláció és hibavizsgálat

A polyfit egyszerű, de magas fokszámon (a rosszul kondicionált Vandermonde-mátrix miatt) instabil lehet — ilyenkor érdemes spline-t vagy Csebisev-alappontokat használni.

## 23. Newton-módszer, szelőmódszer, húrmódszer

Ezek a módszerek az  $f(x) = 0$  nemlineáris egyenlet gyökét közelítik iteratíván, amikor analitikus megoldás nincs.

### Newton-módszer (érintőmódszer)

Az aktuális  $x_k$  pontban felírt érintő zérushelyét vesszük a következő közelítésnek:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)},$$

A Newton-iteráció

$$X_{k+1} = X_k - \frac{f(X_k)}{f'(X_k)}.$$

**Konvergencia.** Ha  $f$  az  $x^*$  egyszeres gyök egy környezetében kétszer folytonosan differenciálható, akkor megfelelő közeli kezdőpontból a módszer **kvadratikus** konvergens; a hibára Taylor-sorfejtésből

$$|e_{k+1}| = \left| \frac{f''(\eta_k)}{2f'(x_k)} \right| e_k^2.$$

Ez azt jelenti, hogy a pontos jegyek száma lépésenként közel **megduplázódik** (pl. az  $f(x) = x^2 - x$  függvényre  $x_0 = 2$ -ből 6 lépésben a pontos 1-et kapjuk). Többváltozós esetben a deriválttal való osztás helyébe a Hesse-mátrix inverze lép:  $x_{k+1} = x_k - H^{-1}(x_k) \nabla f(x_k)$  (ez egyben a Newton-féle optimalizálás).

```
matlab Kód másolása  
  
% Newton-módszer megvalósítása  
f = @(x) x^3 - 2*x - 5; % Függvény  
df = @(x) 3*x^2 - 2; % Deriváltja  
x0 = 2; % Kezdeti közelítés  
tol = 1e-6; % Tűrés  
max_iter = 100;  
  
for k = 1:max_iter  
    x1 = x0 - f(x0)/df(x0); % Newton iteráció  
    if abs(x1 - x0) < tol  
        fprintf('Gyök: %.6f, Iterációk száma: %d\n', x1, k);  
        break;  
    end  
    x0 = x1;  
end
```

MATLAB-megvalósítás

- **Előny:** jó kezdőpontból nagyon gyors (kvadratikus) konvergencia.
- **Hátrány:** deriváltat igényel; rossz kezdőpontból nem garantált a konvergencia.

### Szelőmódszer

A Newton-módszer deriváltmentes változata: az  $f'(x_k)$  helyébe a numerikus derivált (két ponton átmenő szelő meredeksége) lép:

$$x_{k+1} = x_k - f(x_k) \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}.$$

*A szelőmódszer iterációja*

$$X_{k+1} = X_k - \frac{f(X_k)(X_k - X_{k-1})}{f(X_k) - f(X_{k-1})}.$$

Geometriailag  $x_{k+1}$  az  $(x_k, f(x_k))$  és  $(x_{k-1}, f(x_{k-1}))$  pontokon átmenő egyenes és az x-tengely metszéspontja. Konvergenciája **szuperlineáris** (rend  $\approx 1,618$ ), mert a hibára  $|e_k| \leq (2\mu_1/\mu_2)|e_{k-1}||e_{k-2}|$ .

```
matlab Kód másolása

% Szelőmódszer megvalósítása
f = @(x) x^3 - 2*x - 5;
x0 = 2; % Első közelítés
x1 = 3; % Második közelítés
tol = 1e-6;
max_iter = 100;

for k = 1:max_iter
    x2 = x1 - f(x1) * (x1 - x0) / (f(x1) - f(x0)); % Szelő iteráció
    if abs(x2 - x1) < tol
        fprintf('Gyök: %.6f, Iterációk száma: %d\n', x2, k);
        break;
    end
    x0 = x1;
    x1 = x2;
end
```

*MATLAB-megvalósítás*

- **Előny:** nem kell derivált; kevésbé érzékeny a kezdőpontra.
- **Hátrány:** a Newtonnál lassabb (de annál olcsóbb lépésenként).

## Húrmódszer (regula falsi)

A szelőmódszer azon változata, amely megőrzi, hogy a két alappont **közrefogja** a gyököt ( $f(a) \cdot f(b) < 0$ ). Az új pont előjele alapján mindig azt a régi pontot tartjuk meg, amellyel a közrefogás fennmarad:

$$c = b - \frac{f(b)(b - a)}{f(b) - f(a)}.$$

*A húrmódszer közelítése*

```

matlab Kód másolása

% Húrmódszer megvalósítása
f = @(x) x^3 - 2*x - 5;
a = 2; % Intervallum bal széle
b = 3; % Intervallum jobb széle
tol = 1e-6;
max_iter = 100;

for k = 1:max_iter
    c = b - f(b) * (b - a) / (f(b) - f(a)); % Húrmódszer képlet
    if abs(f(c)) < tol
        fprintf('Gyök: %.6f, Iterációk száma: %d\n', c, k);
        break;
    end
    if f(a) * f(c) < 0
        b = c;
    else
        a = c;
    end
end
end

```

MATLAB-megvalósítás

- **Előny:**  $f(a) \cdot f(b) < 0$  esetén **garantáltan** konvergens; nem kell derivált.
- **Hátrány:** lassabb lehet, mert az egyik végpont gyakran változatlan marad.

## Összegzés

| Módszer        | Előnyök                                    | Hátrányok                                 |
|----------------|--------------------------------------------|-------------------------------------------|
| Newton-módszer | Gyors (kvadrátikus konvergencia).          | Derivált szükséges, nem mindig konvergál. |
| Szelőmódszer   | Deriváltmentes, egyszerűbb megvalósítás.   | Lassabb, mint a Newton-módszer.           |
| Húrmódszer     | Garantált konvergencia egy intervallumban. | Lassú konvergencia bizonyos esetekben.    |

A módszerek összehasonlítása

A Newton a leggyorsabb, de deriváltat és jó kezdőpontot kíván; a szelőmódszer deriváltmentes kompromisszum; a húrmódszer a leglassabb, de a közrefogás miatt a legbiztosabb.

## 24. Legkisebb négyzetek módszere

### A feladat

Ha több feltételünk van, mint ismeretlenünk (túlhatározott rendszer), általában nincs minden pontban illeszkedő megoldás. Ekkor azt az  $x$ -et keressük, amely a lehető legkisebb eltérést adja — az eltérések **négyzetösszegét** minimalizálja:

$$i = 1, 2, \dots, n.$$

*Az adathalmaz*

$$S = \sum_{i=1}^n (y_i - f(x_i))^2,$$

*A minimalizálandó négyzetösszeg*

Tipikus eset az  $y = Ax + \varepsilon$  modell, ahol  $\varepsilon$  zaj. A Gauss–Markov-tétel szerint, ha a zaj komponensei független, nulla várható értékű, állandó szórású változók, akkor a legjobb torzítatlan lineáris becslést épp az  $\|y - Ax\|_2$  minimalizálása adja.

### A normálegyenlet

**Tétel.** Ha  $A \in \mathbb{R}^{m \times n}$ , és  $A^H A$  nem szinguláris, akkor  $\|b - Ax\|_2$  minimumát az az  $x^*$  veszi fel, amelyre teljesül a **normálegyenlet**:

$$A^H A x^* = A^H b.$$

$$a = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2},$$
$$b = \frac{\sum y_i - a \sum x_i}{n}.$$

*A normálegyenletek*

A bizonyítás kulcsa, hogy a minimumban a reziduum merőleges  $A$  oszlopterére ( $A^H r^* = 0$ ), és így  $\|r\|_2^2 = \|r^*\|_2^2 + \|r^* - r\|_2^2 \geq \|r^*\|_2^2$ . Az  $A^H A$  mátrix Hermite-szimmetrikus és pozitív szemidefinit; ha  $A$  rangja  $n$ , akkor pozitív definit.

### Egyenesillesztés (lineáris regresszió)

A legegyszerűbb eset az  $f(x) = ax + b$  egyenes illesztése:

$$f(x) = ax + b,$$

*Lineáris modell*

ahol  $a$  a meredekség,  $b$  a tengelymetszet; ezeket a normálegyenletekből kapjuk.

### Megoldási módok és kondíció

- Ha a normálegyenlet jól kondicionált (pl. spline-közelítésnél), megoldható közvetlenül **Cholesky-felbontással** ( $A^H A = LL^H$ ).
- Gyakran azonban a normálegyenlet kondíciószáma a kiindulásinak a **négyzete** — különösen hasonló jellegű alapfüggvényeknél. Ilyenkor stabilabb az  $A$  közvetlen **QR-felbontása**, az  $A^H A$  explicit képzése nélkül.

## MATLAB

A polinomillesztést a legkisebb négyzetek módszerével a **polyfit(x,y,n)** végzi (a  $\sum (p(x_i) - y_i)^2$  minimalizálásával); általános túlhatározott rendszere a backslash ( $A \backslash b$ ) operátor QR-alapú, stabil megoldást ad.

```
matlab Kód másolása

% Adatpontok
x = [1, 2, 3, 4, 5];
y = [2.2, 2.8, 3.6, 4.5, 5.1];

% Egyenes illesztése
p = polyfit(x, y, 1); % 1. fokú polinom illesztése
a = p(1); % Meredekség
b = p(2); % Tengelymetszet

% Kiértékelés és ábrázolás
xx = linspace(min(x), max(x), 100); % Sűrű x-tartomány
yy = polyval(p, xx); % Az illesztett egyenes értékei

plot(x, y, 'ro', 'MarkerSize', 8, 'MarkerFaceColor', 'r'); % Adatpontok
hold on;
plot(xx, yy, 'b-', 'LineWidth', 1.5); % Illesztett egyenes
grid on;
xlabel('x');
ylabel('y');
title('Legkisebb négyzetek módszere - Egyenes illesztése');
legend('Adatpontok', 'Illesztett egyenes');
hold off;
```

Egyenesillesztés MATLAB-ban



```
% Polinom illesztése
p = polyfit(x, y, 2); % 2. fokú polinom
yy_poly = polyval(p, xx); % Kiértékelés

% Ábrázolás
plot(x, y, 'ro', xx, yy_poly, 'g-', 'LineWidth', 1.5);
grid on;
xlabel('x');
ylabel('y');
title('Legkisebb négyzetek módszere - Polinom illesztése');
legend('Adatpontok', 'Illesztett polinom');
```

Polinomillesztés MATLAB-ban

## Az illesztés jósága ( $R^2$ )

A modell magyarázóerejét az  $R^2$  (determinációs együttható) méri (1-hez közeli érték a jó):

$$R^2 = 1 - \frac{\sum (y_i - f(x_i))^2}{\sum (y_i - \bar{y})^2},$$

Az  $R^2$  képlete

```
% Hibaszámítás
SS_res = sum((y - polyval(p, x)).^2); % Reziduális négyzetösszeg
SS_tot = sum((y - mean(y)).^2); % Teljes négyzetösszeg
R2 = 1 - SS_res / SS_tot;

fprintf('R^2 érték: %.4f\n', R2);
```

 $R^2$  számítása MATLAB-ban

- **Előny:** egyszerű, széles körben alkalmazható, jó modell mellett pontos.
- **Hátrány:** a minőség erősen függ a választott függvényformától; nemlineáris illesztésnél a konvergencia nem mindig garantált.

## 25. Spline-közelítés, megvalósítása a MATLAB-ban

### Miért spline?

A magas fokszámú polinominterpoláció nagy adathalmaznál a tartomány szélein erősen oszcillálhat (**Runge-jelenség**). A spline ezt elkerüli: **több, alacsony fokszámú** polinomot illeszt szakaszonként, amelyek a csomópontokban simán csatlakoznak. Így stabil, sima görbét kapunk.

### Matematikai alap

Az  $(x_i, y_i)$  adatpontokhoz olyan darabos  $S(x)$  függvényt keresünk, amely:

$$(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$$

*Az adatpontok*

$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3.$$

*Szakaszonként harmadfokú polinom*

- minden  $[x_i, x_{i+1}]$  intervallumon (köbös spline esetén) **harmadfokú** polinom;
- a csomópontokban **follytonos**, és az  $S'(x)$ ,  $S''(x)$  deriváltak is folytonosak;
- **természetes spline** esetén a végpontokban  $S''(x_0) = S''(x_n) = 0$ .

A folytonossági és illeszkedési feltételek lineáris egyenletrendszert adnak az együtthatókra (eggyel kevesebb feltétel van, mint együttható, ezért kell egy peremfeltétel — pl. a természetes spline, vagy egy előírt derivált). Például a  $(0, 0)$ ,  $(1, -1)$ ,  $(2, 0)$  pontokra illesztett kvadratikus spline (a csatlakozási pontban megegyező első deriválttal, és  $p'_1(0) = 0$  kikötéssel):  $p_1(x) = -x^2$  a  $[0,1]$ -en, és  $p_2(x) = 3x^2 - 8x + 4$  az  $[1,2]$ -n; innen pl.  $f(0,5) \approx -0,25$  és  $f(1,5) \approx -1,25$ .

### Spline a MATLAB-ban

A köbös spline a beépített **spline(x, y, xx)** függvénnyel állítható elő (sokparaméteres közelítés lévén lényegesen jobb approximáció, mint az egyetlen polinom):

```
% Adatpontok
x = [0, 1, 2, 3, 4];
y = [1, 2.5, 0.5, 3.5, 2];

% Spline közelítés
xx = linspace(min(x), max(x), 100); % Sűrűbb x-tartomány az ábrázoláshoz
yy = spline(x, y, xx); % Spline kiértékelése

% Ábrázolás
plot(x, y, 'ro', 'MarkerSize', 8, 'MarkerFaceColor', 'r'); % Adatpontok
hold on;
plot(xx, yy, 'b-', 'LineWidth', 1.5); % Spline görbe
grid on;
xlabel('x');
ylabel('y');
title('Spline közelítés');
legend('Adatpontok', 'Spline közelítés');
hold off;
```

*Természetes (kőbös) spline MATLAB-ban*

Az együttthatókkal közvetlenül is dolgozhatunk: a `pp = spline(x,y)` a darabos polinomot (piecewise polynomial) adja, amelyet a **ppval** értékeli ki; alacsony szintű kezelésre az `mkpp` és `unmkpp` való. Zajos adathoz a **csaps** (simító spline) használható, amely egy paraméterrel hangolja az illeszkedés és a simaság közti egyensúlyt:

```
% Zajos adatok
x = linspace(0, 10, 10);
y = sin(x) + 0.2*randn(size(x)); % Zajos szinusz függvény

% Simító spline
pp = csaps(x, y, 0.9); % 0.9 a simítási paraméter
xx = linspace(min(x), max(x), 100);
yy = fnval(pp, xx);

% Ábrázolás
plot(x, y, 'ro', 'MarkerSize', 8, 'MarkerFaceColor', 'r'); % Zajos adatok
hold on;
plot(xx, yy, 'g-', 'LineWidth', 1.5); % Simító spline görbe
grid on;
xlabel('x');
ylabel('y');
title('Simító spline közelítés');
legend('Adatpontok', 'Simító spline');
hold off;
```

*Simító spline zajos adatokhoz*

## Előnyök és hátrányok

- **Előny:** nagy adathalmazra stabilabb, mint a polinominterpoláció; sima, természetes görbe.
- **Hátrány:** az implementáció bonyolultabb; az eredmény érzékeny lehet a csomópontok eloszlására.

## 26. Numerikus integrálás, kvadrátúra-formulák

### A feladat

A kvadrátúra a numerikus integrálás szinonimája: az

$$I = \int_a^b f(x) dx.$$

*A közelítendő határozott integrál*

határozott integrált közelítjük, amikor a primitív függvény nem áll rendelkezésre, vagy nem is elemi (pl.  $e^{x^2}$ ), illetve ha  $f$  csak táblázatos adatként ismert.

### Kvadrátúra-formulák

A határozott integrált súlyozott függvényérték-összeggel közelítjük:

$$I \approx \sum_{i=1}^n w_i f(x_i),$$

*Az általános kvadrátúra-formula*

$$\int_a^b f(x) dx \approx Q_n(f) = \sum_{i=1}^n w_i f(x_i),$$

ahol az  $x_i \in [a, b]$  a (páronként különböző) **alappontok**, a  $w_i$  a **súlyok**. Mind az integrál, mind  $Q_n$  **homogén és additív** (lineáris) leképezés.

### A formula rendje (pontossága)

A képlethiba  $R_n(f) = \int f - Q_n(f)$ ; a formula **pontos**  $f$ -re, ha ez nulla. A  $Q_n$  **rendje**  $r$ , ha pontos az  $1, x, \dots, x^r$  hatványfüggvényekre, de  $x^{r+1}$ -re már nem. A legalább 0 rendhez a súlyokra  $\sum w_i = b - a$  szükséges.

**Tétel.** Egy  $n$  alappontos kvadrátúra-formula rendje legfeljebb  $2n - 1$ . (A bizonyítás a  $q(x) = \omega_{n-1}(x)^2$   $2n$ -edfokú polinomot tekinti, amelyre a formula nem pontos.)

### Néhány alapformula

$$I \approx \frac{b-a}{2} (f(a) + f(b)).$$

*Trapézszabály*

A **trapézszabály** ( $n = 2$ ) a függvényt lineárisan közelíti:

$$\int_a^b f(x) dx \approx \frac{b-a}{2} (f(a) + f(b)).$$

$$I \approx \frac{b-a}{6} \left( f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right).$$

*Simpson-szabály*

A **Simpson-szabály** másodfokú polinommal, három ponton át:

$$\int_a^b f(x) dx \approx \frac{b-a}{6} \left( f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right).$$

A **Gauss-kvadraturában** az alappontokat és a súlyokat is szabadon választjuk, így  $n$  ponttal elérhető a maximális,  $2n - 1$  rend.

## MATLAB

matlab

 Kód másolása

```
% Trapéz szabály megvalósítása
f = @(x) x.^2; % Példa függvény
a = 0; % Integrál alsó határa
b = 1; % Integrál felső határa

I_trap = (b-a)/2 * (f(a) + f(b)); % Trapéz szabály képlet
fprintf('Trapéz szabállyal számított integrál: %.6f\n', I_trap);
```

*Trapézsabály MATLAB-ban*

matlab

 Kód másolása

```
% Szimpson-szabály megvalósítása
f = @(x) x.^2; % Példa függvény
a = 0; % Integrál alsó határa
b = 1; % Integrál felső határa
m = (a+b)/2; % Középső pont

I_simpson = (b-a)/6 * (f(a) + 4*f(m) + f(b)); % Szimpson-szabály képlet
fprintf('Szimpson-szabállyal számított integrál: %.6f\n', I_simpson);
```

*Simpson-szabály MATLAB-ban*

```

matlab Kód másolása

% Gauss-kvadratura 2 pontos megvalósítása
f = @(x) x.^2; % Példa függvény
a = 0; % Integrál alsó határa
b = 1; % Integrál felső határa

% Gauss-csomópontok és súlyok
x = [-1/sqrt(3), 1/sqrt(3)]; % Csomópontok standard tartományon (-1, 1)
w = [1, 1]; % Súlyok
g = @(t) f((b-a)/2 * t + (a+b)/2) * (b-a)/2; % Átskálázás [a, b]-re

I_gauss = sum(w .* g(x)); % Gauss-kvadratura képlet
fprintf('Gauss-kvadratúrával számított integrál: %.6f\n', I_gauss);

```

Kétpontos Gauss-kvadratura

| Módszer          | Előnyök                                      | Hátrányok                                        |
|------------------|----------------------------------------------|--------------------------------------------------|
| Trapéz szabály   | Egyszerű, könnyen érthető.                   | Lassan konvergál, ha a függvény nem sima.        |
| Szimpson-szabály | Gyorsabb konvergencia, pontosabb eredmények. | Csak páros számú alosztás esetén használható.    |
| Gauss-kvadratura | Nagy pontosság kis számú csomóponttal is.    | Az integrálási pontokat előre ki kell számítani. |

Előnyök és hátrányok

A trapéz- és Simpson-szabály egyszerű és táblázatos adataira is jó; a Gauss-kvadratura ugyanannyi pontból lényegesen pontosabb, ha a függvény sima.

## 27. Interpolációs kvadrátúra-formulák, ezek hibája

### Interpolációs kvadrátúra-formulák

Egy  $Q_n(f) = \sum w_i f(x_i)$  formula **interpolációs**, ha úgy áll elő, hogy az  $f$ -et az alappontokra felírt Lagrange-polinommal helyettesítjük, és azt integráljuk:

$$\int_a^b f(x) dx \approx \int_a^b P_n(x) dx,$$

*Az integrál az interpoláló polinom integráljaként*

$$\int_a^b f(x) dx \approx \sum_{i=0}^n w_i f(x_i),$$

*Az általános alak*

$$\int_a^b f(x) dx \approx \int_a^b p_{n-1}(x) dx = \sum_{i=1}^n f(x_i) \int_a^b L_i(x) dx, \quad w_i = \int_a^b L_i(x) dx.$$

**Tétel (mindkét irány).** Minden  $n$  alappontos interpolációs kvadrátúra-formula rendje **legalább  $n - 1$** ; és megfordítva, ha egy formula rendje legalább  $n - 1$ , akkor interpolációs. (Az utóbbi az  $L_i(x_j) = \delta_{ij}$  tulajdonságon múlik.)

$$\int_a^b f(x) dx \approx \frac{b-a}{2} (f(a) + f(b)).$$

*Trapézszabály ( $n = 1$ )*

$$\int_a^b f(x) dx \approx \frac{b-a}{6} \left( f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right).$$

*Simpson-szabály ( $n = 2$ )*

A trapézszabály lineáris, a Simpson-szabály másodfokú interpoláló polinomhoz tartozik; a Gauss-kvadrátúra az alappontokat is optimálisan választja (rend  $2n - 1$ ).

### Hibaanalízis

Ha  $f$  az  $[a, b]$ -n  $n$ -szer folytonosan differenciálható, a Lagrange-interpoláció hibakorlátjából ( $\omega_{n-1}(x) = \prod (x-x_j)$ ):

$$E(f) = \int_a^b f(x) dx - \sum_{i=0}^n w_i f(x_i).$$

*Az általános hibatag*



$$E(f) \propto \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i),$$

A derivált-alapú hiba

$$R_n(f) = \int_a^b \frac{\omega_{n-1}(x)}{n!} f^{(n)}(\xi) dx, \quad |R_n(f)| \leq \frac{\|f^{(n)}\|_\infty}{n!} \int_a^b |\omega_{n-1}(x)| dx.$$

Ha  $\omega_{n-1}$  jeltartó, a középértéktétellel  $R_n(f) = f^{(n)}(\eta) \cdot \int_a^b \omega_{n-1}(x)/n! dx$  is felírható. A hibakorlát akkor a legkisebb, ha  $\int_a^b |\omega_{n-1}(x)| dx$  minimális:

**Állítás.** Az  $\int_a^b |\omega_{n-1}(x)| dx$  pontosan akkor minimális, ha alappontoknak a másodfajú **Csebisev-polinomok** ( $U_0 = 1$ ,  $U_1 = 2x$ ,  $U_n = 2xU_{n-1} - U_{n-2}$ ) zérushelyeit választjuk — ez tompítja a Runge-féle szélső oszcillációt.

A konvergencia rendje az egyes módszerekre (lépésközzel,  $h$ ): trapéz  $O(h^2)$ , Simpson  $O(h^4)$ , Gauss-kvadratura  $O(h^{2n})$  sima függvényre.

## MATLAB

matlab

 Kód másolása

```
% Függvény és intervallum
f = @(x) x.^2; % Példa függvény
a = 0; b = 1; % Integrál határai
I_exact = integral(f, a, b); % Pontos integrál MATLAB "integral" függvényével

% Trapéz szabály
I_trap = (b-a)/2 * (f(a) + f(b));
error_trap = abs(I_exact - I_trap);

fprintf('Trapéz szabály hibája: %.6f\n', error_trap);
```

Trapézsabály hibavizsgálata

matlab

 Kód másolása

```
% Simpson-szabály
m = (a+b)/2;
I_simpson = (b-a)/6 * (f(a) + 4*f(m) + f(b));
error_simpson = abs(I_exact - I_simpson);

fprintf('Simpson-szabály hibája: %.6f\n', error_simpson);
```

Simpson-szabály hibavizsgálata

```
% Gauss-kvadratúra 2 pontos
x = [-1/sqrt(3), 1/sqrt(3)];
w = [1, 1];
g = @(t) f((b-a)/2 * t + (a+b)/2) * (b-a)/2;
I_gauss = sum(w .* g(x));
error_gauss = abs(I_exact - I_gauss);

fprintf('Gauss-kvadratúra hibája: %.6f\n', error_gauss);
```

Gauss-kvadratúra hibavizsgálata

| Módszer          | Hibája      | Előnyök                            | Hátrányok                                   |
|------------------|-------------|------------------------------------|---------------------------------------------|
| Trapéz szabály   | $O(h^2)$    | Egyszerű, könnyen érthető.         | Lassan konvergál.                           |
| Szimpson-szabály | $O(h^4)$    | Pontosabb, gyorsabban konvergál.   | Csak páros számú osztás esetén használható. |
| Gauss-kvadratúra | $O(h^{2n})$ | Nagy pontosság kevés csomóponttal. | Bonyolultabb megvalósítás.                  |

Összegzés

## 28. Véges differenciák, Newton-Cotes formulák

### Véges differenciák

Ekvidisztáns (egyenlő,  $h = x_{i+1} - x_i$  lépésközű) alappontokon a függvényértékek különbségeivel közelítünk deriváltakat és építünk interpolációs polinomot. Az  $i$ -edrendű véges differenciát kettős rekurzió definiálja:

$$\Delta^0 f_k = f_k, \quad \Delta^i f_k = \Delta^{i-1} f_{k+1} - \Delta^{i-1} f_k.$$

A deriváltak közelítései:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}.$$

*Első rendű előre differencia*

$$f'(x) \approx \frac{f(x) - f(x-h)}{h}.$$

*Első rendű hátra differencia*

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}.$$

*Középponti differencia (másodrendű pontosság)*

$$f''(x) \approx \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}.$$

*Másodrendű derivált középponti differenciával*

Az előre és hátra differencia  $O(h)$  pontosságú, a szimmetrikus **középponti** differencia  $O(h^2)$ .

### A Newton–Gregory-interpolációs polinom

A binomiális együttható általánosításával —  $(t \text{ alatt } j) = t(t-1)\dots(t-j+1)/j!$ , ahol  $t = (x - x_0)/h$  — a Lagrange-interpolációs polinom véges differenciákkal tömören felírható:

$$p_{n-1}(x_0 + th) = \sum_{i=0}^{n-1} \binom{t}{i} \Delta^i f_0.$$

Az  $(t \text{ alatt } j) = ((t-j+1)/j) \cdot (t \text{ alatt } j-1)$  rekurzióval a helyettesítési érték  $O(n)$  művelettel számítható (ha a  $\Delta^i f_0$  ismert). *Példa:* az  $f(x) = x^2 - x$  függvényre a 0, 1, 2 pontokon  $\Delta^2 f_0 = 2$ , és a polinom  $2 \cdot (t \text{ alatt } 2) = t^2 - t = x^2 - x$  — visszkapjuk az eredetit.

### Newton–Cotes formulák

A Newton–Cotes formulák az interpolációs kvadratura-formulák azon (régi) osztálya, amely **ekvidisztáns** alappontokat használ. Ha az integrálási határok alappontok, **zárt**, ha nem, **nyitott** formuláról beszélünk.

$$\int_a^b f(x) dx \approx \sum_{i=0}^n w_i f(x_i),$$

*Az általános formula*

A leggyakoribb zárt formulák:

$$\int_a^b f(x) dx \approx \frac{b-a}{2} (f(a) + f(b)).$$

*Trapézszabály ( $n = 1$ )*

$$\int_a^b f(x) dx \approx \frac{b-a}{6} \left( f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right).$$

*Simpson-szabály ( $n = 2$ )*

$$\int_a^b f(x) dx \approx \frac{3(b-a)}{8} (f(a) + 3f(a+h) + 3f(b-h) + f(b)),$$

*Simpson 3/8 szabály ( $n = 3$ )*

$$\int_a^b f(x) dx \approx \frac{2(b-a)}{45} (7f(a) + 32f(a+h) + 12f(a+2h) + 32f(a+3h) + 7f(b)),$$

*Boole-szabály ( $n = 4$ )*

## Hibaanalízis

A Newton–Cotes formulák hibája a magasabb rendű deriváltaktól függ:

$$E(f) \propto \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i),$$

*A hibatag*

A konvergencia rendje (lépésközzel): trapéz  $O(h^2)$ , Simpson  $O(h^4)$ , Simpson 3/8 szintén  $O(h^4)$ , Boole  $O(h^6)$ . Magasabb fokszámon a Newton–Cotes súlyok előjelet váltanak (és nőnek), ezért a gyakorlatban inkább **összetett** (szakaszonként alkalmazott) formulákat használunk.

## MATLAB

matlab

 Kód másolása

```
% Véges differenciák példája
f = @(x) sin(x); % Függvény
x = pi/4; % Deriválási pont
h = 0.01; % Lépésköz

% Első derivált közelítése
f_prime_forward = (f(x+h) - f(x)) / h; % Előre differencia
f_prime_backward = (f(x) - f(x-h)) / h; % Hátrafelé differencia
f_prime_central = (f(x+h) - f(x-h)) / (2*h); % Középpontos differencia

fprintf('Előre differencia: %.6f\n', f_prime_forward);
fprintf('Hátrafelé differencia: %.6f\n', f_prime_backward);
fprintf('Középpontos differencia: %.6f\n', f_prime_central);
```

*Véges differenciák MATLAB-ban*

matlab

 Kód másolása

```
% Trapéz szabály
f = @(x) x.^2; % Függvény
a = 0; b = 1; % Intervallum
I_trap = (b-a)/2 * (f(a) + f(b));
fprintf('Trapéz szabály eredménye: %.6f\n', I_trap);

% Simpson-szabály
m = (a+b)/2; % Középpont
I_simpson = (b-a)/6 * (f(a) + 4*f(m) + f(b));
fprintf('Simpson-szabály eredménye: %.6f\n', I_simpson);

% Simpson 3/8 szabály
h = (b-a)/3;
I_simpson_38 = (3*h/8) * (f(a) + 3*f(a+h) + 3*f(b-h) + f(b));
fprintf('Simpson 3/8 szabály eredménye: %.6f\n', I_simpson_38);
```

*Newton–Cotes formulák MATLAB-ban*

## 29. Numerikus integrálás a MATLAB-ban

A MATLAB dedikált, hatékony függvényeket kínál határozott integrálok numerikus kiszámítására. A legfontosabbak: **integral** (adaptív kvadratura általános integrálokhoz), **trapz** (trapézsabály táblázatos adatra), valamint a régebbi **quad** / **quadl** (ma már elavultak, helyettük az **integral** ajánlott).

### Az **integral** függvény

Modern, adaptív eljárás, amely az  $[a, b]$  intervallumon automatikusan finomítja a felosztást a kívánt pontosságig:

matlab

Kód másolása

```
q = integral(fun, a, b);
```

*Az **integral** szintaxisa*

A **fun** az integrálandó függvény (jellemzően anonim függvény, pl.  $@(x) x.^2$ ), az **a** és **b** az integrál határai.

matlab

Kód másolása

```
f = @(x) x.^2; % Függvény
a = 0; % Alsó határ
b = 1; % Felső határ

I = integral(f, a, b); % Integrál kiszámítása
fprintf('integral függvénnyel számított integrál: %.6f\n', I);
```

*Példa az **integral** használatára*

### A **trapz** függvény

Trapézsabály alapján integrál; különösen **táblázatos** (mért) adatra hasznos, amikor csak az osztáspontok és a függvényértékek ismertek:

matlab

Kód másolása

```
q = trapz(x, y);
```

*A **trapz** szintaxisa*

matlab

 Kód másolása

```
x = linspace(0, 1, 5); % Osztaspontok
y = x.^2; % Függvényértékek

I_trapz = trapz(x, y); % Trapékszabály alapján számított integrál
fprintf('trapz függvénnyel számított integrál: %.6f\n', I_trapz);
```

*Példa a trapz használatára*

## Többdimenziós integrálás

A két- és háromdimenziós integrálokra az **integral2**, illetve **integral3** való:

matlab

 Kód másolása

```
q = integral2(fun, xmin, xmax, ymin, ymax);
```

*Az integral2 szintaxisa*

matlab

 Kód másolása

```
f = @(x, y) x.^2 + y.^2;

I2 = integral2(f, 0, 1, 0, 1); % Kétdimenziós integrál
fprintf('integral2 függvénnyel számított integrál: %.6f\n', I2);
```

*Példa:  $f(x,y) = x^2 + y^2$  a  $[0,1] \times [0,1]$  tartományon*

## Saját kvadratura és hibakezelés

Saját módszer (pl. Gauss-kvadratura) is könnyen megvalósítható:

matlab

 Kód másolása

```
% Függvény
f = @(x) exp(-x.^2);

% Gauss-kvadratura csomópontok és súlyok
x = [-1/sqrt(3), 1/sqrt(3)]; % Standard [-1, 1] tartomány
w = [1, 1]; % Súlyok
a = 0; b = 1; % Intervallum

% Átskálázás és integrálás
g = @(t) f((b-a)/2*t + (a+b)/2) * (b-a)/2;
I_gauss = sum(w .* g(x));
fprintf('Gauss-kvadraturával számított integrál: %.6f\n', I_gauss);
```

A numerikus integrálás tipikus buktatói:

- **Szakadás:** a függvény diszkontinuitásánál az **adaptív** integral jobban teljesít, mint a trapz.
- **Gyorsan változó (rossz kondíciójú) függvény:** növelni kell az osztópontok számát.
- **Nagyon nagy vagy kicsi tartomány:** átskálázás javíthatja a pontosságot.

| Módszer                                         | Előnyök                            | Hátrányok                            |
|-------------------------------------------------|------------------------------------|--------------------------------------|
| <code>integral</code>                           | Adaptív, gyors, általános célú.    | Korlátozott szabályozhatóság.        |
| <code>trapz</code>                              | Táblázatos adatokkal jól működik.  | Lassú konvergencia (Trapéz szabály). |
| <code>integral2</code> , <code>integral3</code> | Egyszerű többdimenziós integrálás. | Lassabb nagyobb dimenziókban.        |

Összegzés